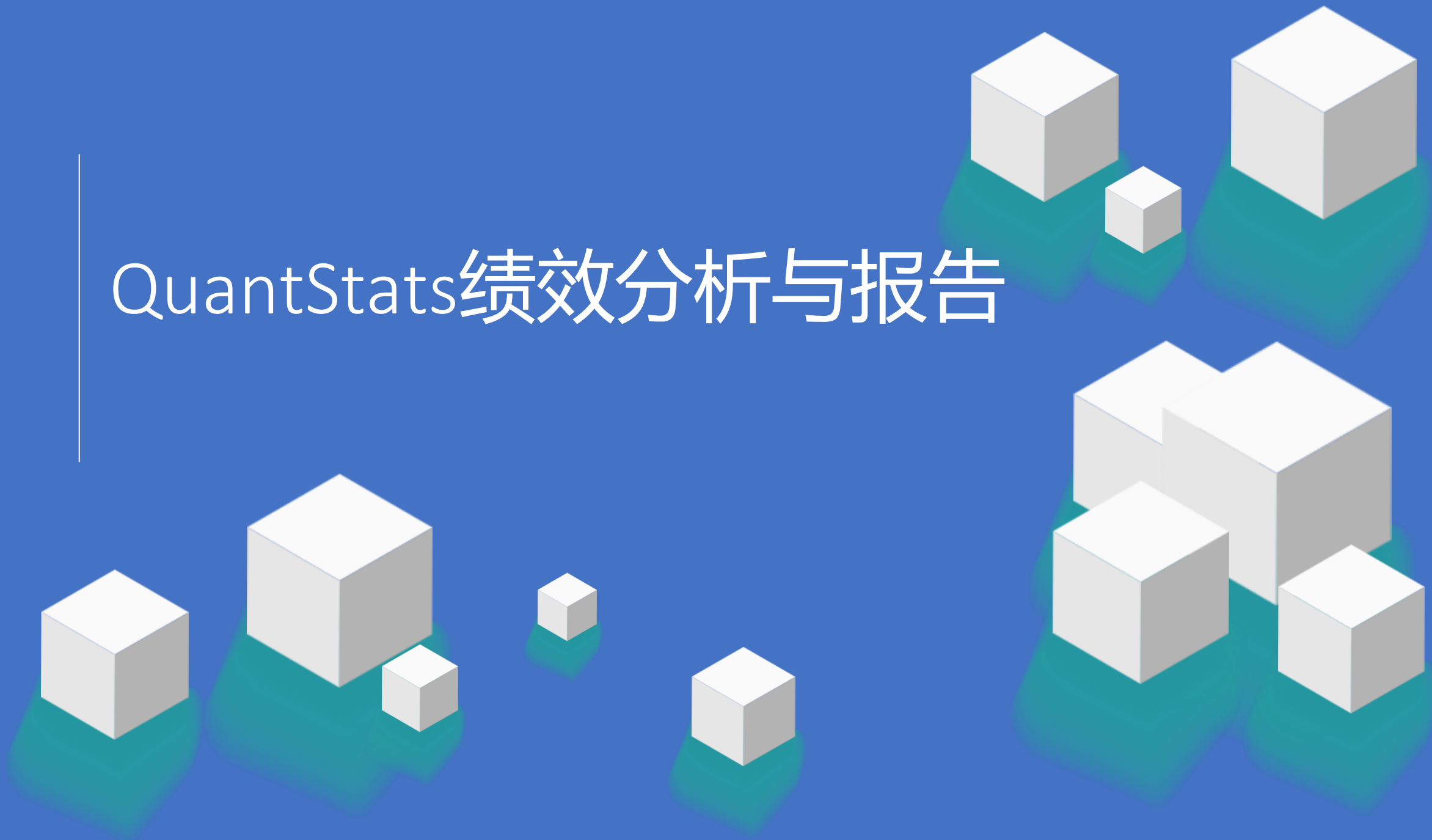


# QuantStats绩效分析与报告



# >> 今天的学习目标

---

## QuantStats绩效分析与报告

- QuantStats 指标体系

CASE: QuantStats绩效分析

Backtrader 内置指标 vs QuantStats 对比

多策略 QuantStats 对比

HTML 报告生成 (英文版 + 中文版)

CASE: 实盘交易绩效分析

CASE: 实盘交易绩效分析Plus

- SVD矩阵分解与因子分析
- 矩阵分解的应用场景
- 在推荐系统、模型微调、图像压缩中的应用
- 在金融市场定价中的应用

场景: 收益率矩阵 SVD 分解

场景: 滚动 SVD -- 市场状态监控

场景: 残差分析 -- 发现有 Alpha 的个股

SVD因子降维

# QuantStats 指标体系

# QuantStats 指标体系

---

## Thinking: QuantStats 是什么?

QuantStats 是面向 Python 的绩效分析与可视化库，建立在 pandas 与 matplotlib 等生态之上。

目标是将策略回测或实盘产生的收益率序列作为统一输入，一次性输出：

- 收益、风险、风险调整收益、分布与胜败统计等多类指标；
- 累计收益、回撤、滚动指标、月度热力图等图表；
- 可独立打开的 HTML 绩效报告，便于存档与对外沟通。

与在策略代码里手写指标相比，QuantStats 的优势在于：口径统一、可复现、与业界报告习惯一致

# CASE: QuantStats 绩效分析

---

## TO DO: 量化投资经理的一天

你是某私募基金的量化投资经理，管理着一个以 A 股为主的量化投资组合。上周，你用 Backtrader 开发并回测了 3 个策略——双均线策略 (DoubleMA)、MACD 策略 和 RSI 策略，标的均为贵州茅台 (600519.SH)，回测区间 2024-01-01 至 2025-12-31。

今天早上，领导让你在下午的投决会上提交一份专业的绩效分析报告，要求涵盖收益、风险、回撤、风险调整收益等维度，并且附上可直接发送给客户的 HTML 可视化报告。

# QuantStats 使用

---

- 安装 QuantStats

`pip install quantstats`

- 核心 API:

`qs.stats`: 数值型绩效指标: 夏普、索提诺、最大回撤、VaR/CVaR、胜率等

`qs.plots`: 各类绩效图: 累计收益、回撤水下、分布、滚动指标等

`qs.reports`: 一键生成完整 HTML 报告 (内部会组合统计与图形)

日常分析可先用 `qs.stats` 得到核心数字, 再用 `qs.reports.html()` 生成对外报告

# QuantStats 指标

---

- 收益类 (Return)

total\_return: 样本期内总收益率 (复利累积)

cagr: 年化复合增长率 (CAGR)

best\_day: 单日最大收益

worst\_day: 单日最大亏损

- 风险类 (Risk)

volatility: 收益率波动率 (通常年化)

max\_drawdown: 最大回撤 (由净值高点至后续低点的最大跌幅)

var\_95: 95% 置信度下的 VaR

cvar\_95: 95% 置信度下的 CVaR / ES

# VaR 与 CVaR 解读

---

- VaR (Value at Risk, 在险价值)

在给定置信水平下（通常 95%），策略在单个交易日内可能遭受的最大亏损。

$\text{VaR}_{95\%}$  = 收益率序列的第 5 百分位数

Thinking: 如果双均线策略的  $\text{VaR}(95\%) = -1.82\%$ ，意味着什么？

在正常市场条件下，有 95% 的把握单日亏损不会超过 1.82%。

=> 换个角度，大约每 20 个交易日 ( $1/20 = 5\%$ )，可能出现一次超过 1.82% 的单日亏损。



# VaR 与 CVaR 解读

- CVaR (Conditional VaR, 条件在险价值)

当损失超过 VaR 阈值后, 超额损失部分的平均值。也称为 期望尾部损失 (Expected Shortfall)。

$$\text{CVaR}_{95\%} = E[\text{Loss} \mid \text{Loss} > \text{VaR}_{95\%}]$$

Thinking: 双均线策略的 VaR(95%) = -1.82%, CVaR(95%) = -2.67%, 意味着什么?

一旦单日亏损突破了 1.82% 的 VaR 边界, 平均亏损幅度会达到 2.67%。

VaR 回答的问题是: "坏到什么程度算 95% 的边界?" —— 它划了一条线。

CVaR 回答的问题是: "一旦坏过界, 平均会坏多少?" —— 它描述了线以外的世界。

对于风控而言, CVaR 比 VaR 更重要, 因为真正致命的不是刚好触及边界, 而是越界之后的深度。

# QuantStats 指标

---

- 风险调整收益 (Risk-adjusted)

sharpe: 夏普比率, 超额收益 per 单位总波动

sortino: 索提诺比率, 仅对下行波动惩罚

calmar: 卡玛比率,  $\text{CAGR} / |\text{最大回撤}|$

omega: Omega 比率, 收益分布上方与下方矩的比值型刻画

gain\_to\_pain: Gain-to-Pain, 盈利与痛苦 (亏损) 之比的常用变体

- 分布特征 (Distribution)

skew: 偏度, 收益分布是否左偏/右偏

kurtosis: 峰度, 尾部厚薄 (极端日是否更频繁)

# QuantStats 指标

---

- 胜败与盈亏指标

win\_rate: 日胜率, 收益为正的交易日占比

profit\_factor: 利润因子, 总盈利 / 总亏损 (绝对值)

payoff\_ratio: 盈亏比, 平均盈利 / 平均亏损 (绝对值)

avg\_win: 平均盈利, 盈利日的平均收益率

avg\_loss: 平均亏损, 亏损日的平均收益率

consecutive\_wins: 最大连胜, 连续盈利的最长交易日天数

consecutive\_losses: 最大连亏, 连续亏损的最长交易日天数

# QuantStats 指标

---

- 相对基准指标

Alpha: 策略相对基准的超额收益 (CAPM 截距项)

Beta: 策略对基准的系统性风险敞口

information\_ratio: 信息比率, 超额收益 / 跟踪误差

tracking\_error: 跟踪误差, 策略收益与基准收益之差的标准差

# CASE: QuantStats 绩效分析

---

Thinking: 从 Backtrader 到 QuantStats, 搭建的流程是怎样的?

搭建完整链路: Backtrader 回测 --> 提取净值序列 --> 转换为日收益率 --> QuantStats 全量分析。

从 Backtrader 的回测引擎中提取每日净值 (NAV), 将其转换为 pandas Series 并计算百分比收益率, 然后传入 QuantStats 进行指标计算和报告生成。

# CASE: QuantStats绩效分析

```
import quantstats as qs
from report_engine import (
    nav_to_returns, backtrader_nav_to_series,
    calc_quantstats_metrics, print_metrics_table,
    generate_html_report, generate_chinese_report
)
from data_loader import run_and_report, INITIAL_CASH

# ---- 第一步：运行 Backtrader 回测 ----
result = run_and_report(
    DoubleMAStrategy, '600519.SH', '2024-01-01', '2025-12-31',
    label='双均线策略', plot=True
)

# ---- 第二步：提取净值 -> 转收益率 ----
nav_series = backtrader_nav_to_series(result['nav'], INITIAL_CASH)
returns = nav_to_returns(nav_series)
```

```
# ---- 第三步：QuantStats 全量指标计算 ----
metrics = calc_quantstats_metrics(returns,
    benchmark=benchmark_returns)
print_metrics_table(metrics, '双均线策略 - QuantStats 全量指标')

# ---- 第四步：生成 HTML 报告（英文版 + 中文版） ----
generate_html_report(
    returns, benchmark=benchmark_returns,
    title='DoubleMA Strategy - QuantStats Report',
    output_path='outputs/1-QuantStats入门/QuantStats报告_双均线_英文.html'
)
generate_chinese_report(
    returns, benchmark=benchmark_returns,
    title='双均线策略 - 绩效分析报告',
    output_path=zh_path
)
```

# CASE：QuantStats绩效分析

=====

场景: 量化投资经理的一天

=====

背景:

你是某私募基金的量化投资经理, 管理着一个以A股为主的量化组合。

上周, 你用 Backtrader 开发了3个策略(双均线/MACD/RSI), 分别在贵州茅台上做了回测。

今天领导要你提交一份专业的绩效分析报告。

=====

第一部分: QuantStats 绩效分析入门

=====

[1] 运行 Backtrader 回测: 双均线策略 600519.SH

双均线策略 | 600519.SH | 2024-01-15 ~ 2025-12-31 | 473个交易日

总收益: +8.52% | 年化: +4.38% | 最大回撤: 12.35% | 夏普: 0.42 | 卡玛: 0.35

交易: 8次 | 胜率: 62.5%

净值序列: 473 个交易日

起始净值: 1.0000

最终净值: 1.0852

=====

双均线策略 - QuantStats 全量指标

=====

|              |         |
|--------------|---------|
| 总收益率         | 8.52%   |
| 年化收益率(CAGR)  | 4.38%   |
| 年化波动率        | 18.23%  |
| 最大回撤         | -12.35% |
| 夏普比率         | 0.4215  |
| 索提诺比率        | 0.6328  |
| 卡玛比率         | 0.3547  |
| Omega比率      | 1.0892  |
| VaR(95%)     | -1.82%  |
| CVaR(95%)    | -2.67%  |
| 偏度(Skew)     | -0.2341 |
| 峰度(Kurtosis) | 1.8756  |
| 日胜率          | 52.43%  |
| 利润因子         | 1.0892  |
| ...          |         |

# Backtrader 内置指标 vs QuantStats 对比

Backtrader 内置的 analyzers 可以计算夏普比率、最大回撤等基础指标，但覆盖面有限。

QuantStats 提供了更完整的 30+ 指标体系和 HTML 报告。

| 对比维度    | Backtrader Analyzers | QuantStats          |
|---------|----------------------|---------------------|
| 设计目的    | 回测引擎内嵌分析，轻量快速        | 专业绩效分析库，指标全面        |
| 夏普口径    | 年化标准差直接除             | 日频收益 * sqrt(252) 年化 |
| 尾部风险    | 不支持 VaR / CVaR       | 支持 VaR、CVaR、偏度、峰度   |
| HTML 报告 | 不支持                  | 一键生成交互式 HTML        |



# Backtrader 内置指标 vs QuantStats 对比

对比输出示例：

| 指标   | Backtrader | QuantStats | 差异说明                  |
|------|------------|------------|-----------------------|
| 总收益率 | 8.52%      | 8.52%      | 一致                    |
| 夏普比率 | 0.4200     | 0.4215     | BT用年化/QS用日频*sqrt(252) |
| 最大回撤 | 12.35%     | 12.35%     | 一致                    |

[QuantStats 独有指标]

索提诺比率: 0.6328  
卡玛比率: 0.3547  
Omega比率: 1.0892  
VaR(95%): -1.82%  
CVaR(95%): -2.67%

夏普比率存在微小差异是因为年化方式不同。Backtrader 直接对年化收益和年化标准差做除法，而 QuantStats 先在日频维度计算，再乘以 sqrt(252) 年化。两种口径在业界都被广泛使用，关键是统一对比口径。

# 多策略 QuantStats 对比

```
# 定义策略列表
strategies = [
    (DoubleMAStrategy, '双均线'),
    (MACDStrategy, 'MACD'),
    (RSIStrategy, 'RSI'),
]

all_metrics = {}
for StrategyClass, label in strategies:
    result = run_and_report(
        StrategyClass, '600519.SH', '2024-01-01', '2025-12-31',
        label=label, plot=False
    )
    nav_series = backtrader_nav_to_series(result['nav'], INITIAL_CASH)
    returns = nav_to_returns(nav_series)
    all_metrics[label] = calc_quantstats_metrics(returns, benchmark=benchmark_returns)
```

```
# 打印多策略对比表
print_comparison_table(all_metrics)
```

# 多策略 QuantStats 对比

对比结果:

| 策略    | 总收益     | 年化     | 最大回撤    | 夏普     | 索提诺    | 卡玛     | 日胜率    |
|-------|---------|--------|---------|--------|--------|--------|--------|
| ----- |         |        |         |        |        |        |        |
| 双均线   | +8.52%  | +4.38% | -12.35% | 0.4215 | 0.6328 | 0.3547 | 52.43% |
| MACD  | +5.21%  | +2.67% | -15.42% | 0.2103 | 0.3156 | 0.1732 | 51.28% |
| RSI   | +12.87% | +6.62% | -9.78%  | 0.5842 | 0.8921 | 0.6769 | 53.62% |

RSI 策略在所有维度表现最优：总收益最高 (+12.87%)、最大回撤最小 (-9.78%)、夏普和卡玛比率均领先

双均线策略居中，收益 / 风险比适中

MACD 策略收益最低且回撤最大，夏普仅 0.21，需考虑参数优化或弃用

# HTML 报告生成 (英文版 + 中文版)

---

TO DO: 制作HTML报告

QuantStats 原生渲染图表时不支持中文字体, 强行通过 monkey-patch 中文化会导致图表中的中文显示为乱码。

因此我们采用双版本方案:

- 英文版: 调用 `generate_html_report()`, 直接使用 QuantStats 原版一键生成英文 HTML 报告, 图表与文字均为英文, 无乱码问题
- 中文版: 调用 `generate_chinese_report()`, 由 `report_engine.py` 自主使用 matplotlib 渲染所有图表 (配置了 SimHei / Microsoft YaHei 等中文字体), 所有标题、标签、指标名称均为中文, 图表以 base64 嵌入 HTML, 不依赖 QuantStats 的渲染管线

# HTML 报告生成 (英文版 + 中文版)

```
# 与脚本一致: OUTPUT_DIR = 'outputs/1-QuantStats入门'
OUTPUT_DIR = 'outputs/1-QuantStats入门'
for name in strategies:
    nav = nav_dict[name]
    ret = nav_to_returns(nav)

    # 英文版: QuantStats 原版一键生成
    en_path = f'{OUTPUT_DIR}/QuantStats报告_{name}_英文.html'
    generate_html_report(ret, benchmark=benchmark_returns,
                        title=f'{name} Strategy - QuantStats Report',
                        output_path=en_path)

    # 中文版: 自主渲染, 中文字体无乱码
    zh_path = f'{OUTPUT_DIR}/QuantStats报告_{name}.html'
    generate_chinese_report(ret, benchmark=benchmark_returns,
                           title=f'{name}策略 - 绩效分析报告',
                           output_path=zh_path)
```

generate\_chinese\_report() 生成的中文版

报告包含以下图表和数据表:

- 累计收益曲线 (含基准对比)
- 水下图 (回撤曲线)
- 月度收益热力图
- 日收益分布直方图
- 滚动夏普比率 / 滚动波动率
- 年度收益柱状图
- 绩效指标明细表
- 月度收益明细表
- 最大回撤 Top 5 表

所有图表均使用 matplotlib 直接渲染为 PNG 并以 base64 编码嵌入 HTML, 确保中文显示正常。

1-QuantStats绩效分析.py

# 数据导入：从 CSV 重新分析

在实际工作中，你可能需要对历史保存的净值数据重新进行分析。

QuantStats 支持从 CSV 文件导入净值序列，无需重新运行回测。

```
import pandas as pd
# 从 CSV 加载净值序列（与脚本导出列一致：date, nav）
nav_df = pd.read_csv('outputs/1-QuantStats入门/双均线策略_每日净值.csv')
nav_df['date'] = pd.to_datetime(nav_df['date'])
nav_df.set_index('date', inplace=True)
nav_series = nav_df['nav']

# 转换为收益率
returns = nav_to_returns(nav_series)

# 重新计算全量指标
metrics = calc_quantstats_metrics(returns, benchmark=benchmark_returns)
print_metrics_table(metrics, '从CSV重新加载 - 双均线策略指标')
```

验证输出：

=====

从CSV重新加载 - 双均线策略指标

=====

|             |         |
|-------------|---------|
| 总收益率        | 8.52%   |
| 年化收益率(CAGR) | 4.38%   |
| 最大回撤        | -12.35% |
| 夏普比率        | 0.4215  |

[验证] 与实时计算结果完全一致

# Summary

---

QuantStats 六大核心能力：

- 全量指标计算：30+ 指标一键输出，覆盖收益 / 风险 / 风险调整 / 分布 / 胜败 / 基准对比六大维度
- 多策略横向对比：将多个策略的指标放在同一张表中，快速识别优劣
- HTML 报告生成：一键产出专业级可视化报告，适合发送给领导和客户
- 中文本地化：采用双版本方案 -- 英文版由 QuantStats 原生生成，中文版由 `generate_chinese_report()` 自主渲染，所有图表中文无乱码
- CSV 数据兼容：支持从文件导入净值，无需重复回测
- 与 Backtrader 无缝衔接：从回测到分析一条龙，工作流连贯

# Summary

---

Thinking: QuantStats 的使用场景是什么?

- 策略回测分析：回测完成后快速获取全量绩效指标，评估策略质量
- 策略 PK：多策略横向对比，基于量化指标做出选择决策
- 客户报告：生成 HTML 报告附件，向客户展示策略表现
- 风控监控：关注 VaR / CVaR / 最大回撤等风险指标，设定预警线
- 实盘复盘：将实盘交易净值导入 QuantStats，定期复盘策略表现



# SVD矩阵分解与因子分析

# 矩阵分解的应用场景

---

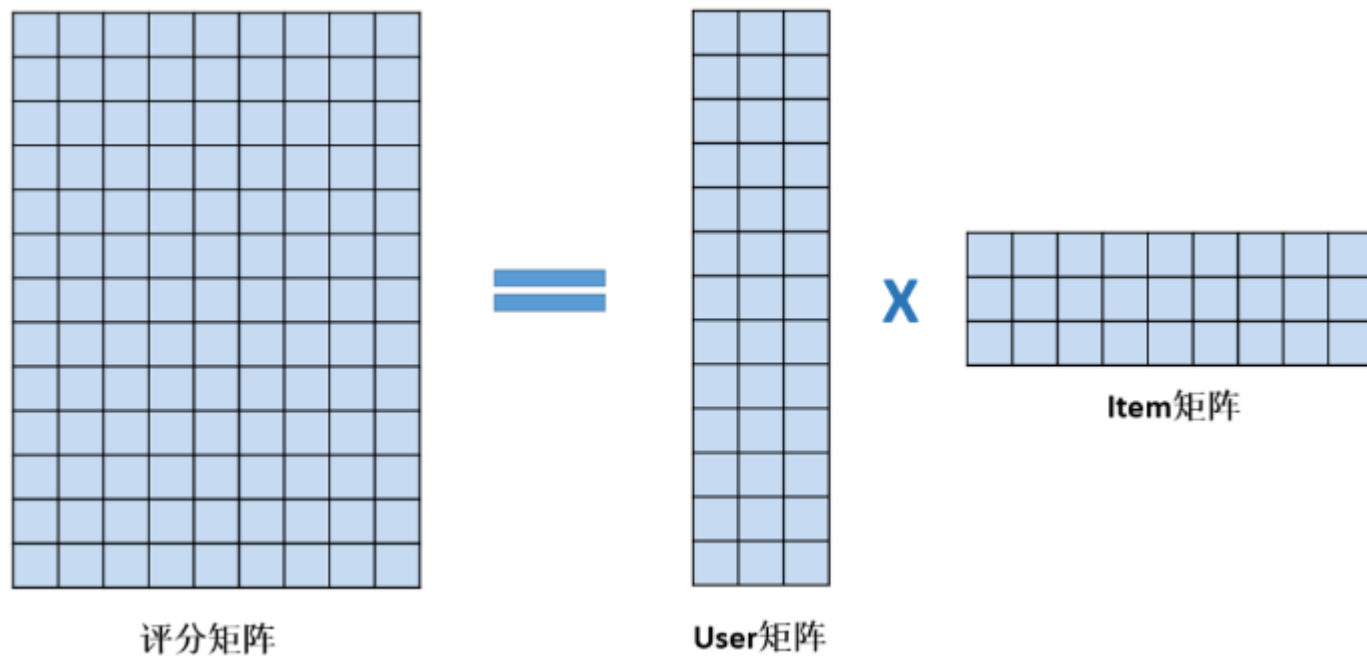
矩阵分解，因子挖掘的使用场景：

- 金融场景中的资产定价，组合优化，信用评分，风险分析
- 模型的高效微调 => LoRA
- 猜你喜欢 => 推荐系统
- 图像压缩、图像修复、去噪 => 计算机视觉



# 什么是矩阵分解

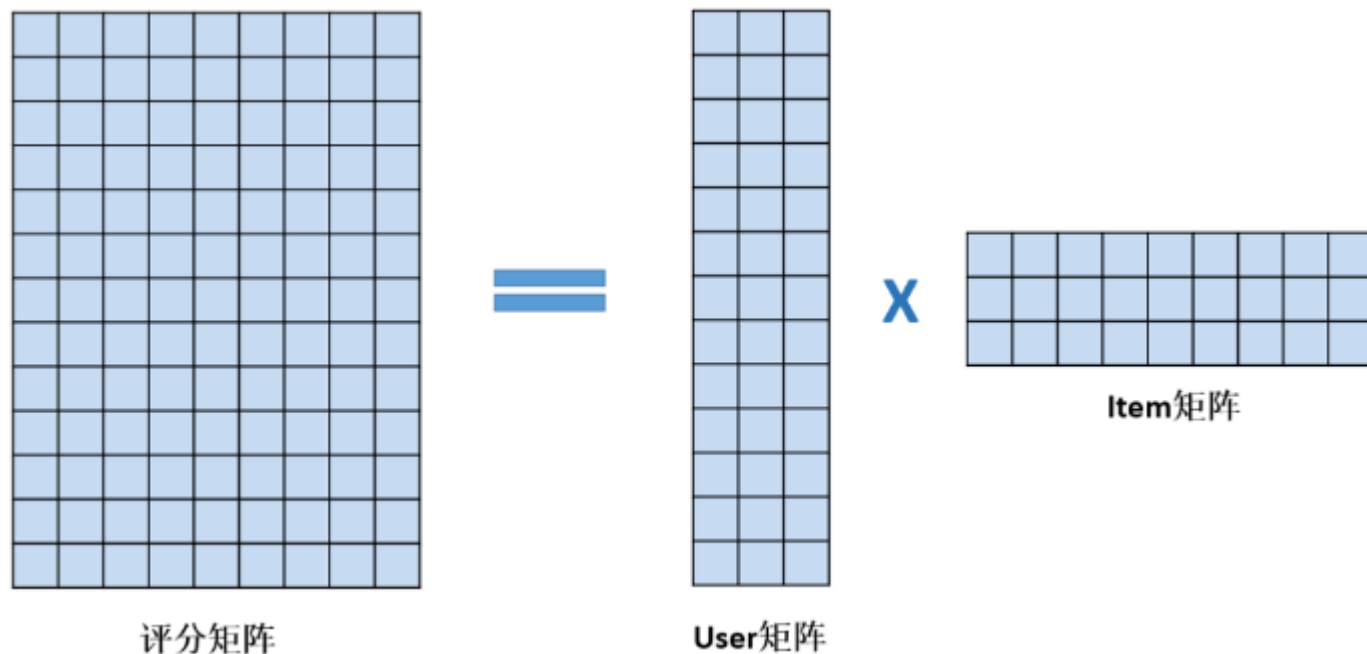
- 矩阵分解要做的是预测出矩阵中缺失的评分，使得预测评分能反映用户的喜欢程度
- 可以把预测评分最高的前K个电影推荐给用户了。



# 什么是矩阵分解

如何从评分矩阵中分解出User矩阵和Item矩阵

- 只有左侧的评分矩阵R是已知的
- User矩阵和Item矩阵是未知
- 学习出User矩阵和Item矩阵，使得User矩阵\*Item矩阵与评分矩阵中已知的评分差异最小 => 最优化问题



# 什么是矩阵分解

- 观察User矩阵：用户的听歌爱好体现在User向量上
- 观察Item矩阵，电影的风格也会体现在Item向量上
- MF用user向量和item向量的内积去拟合评分矩阵中该user对该item的评分，内积的大小反映了user对item的喜欢程度。内积大匹配度高，内积小匹配度低。
- 隐含特征个数k，k越大，隐类别分得越细，计算量越大。

|         | 动作    | 动画    | 爱情    |
|---------|-------|-------|-------|
| user 1  | 0.93  | -0.08 | 0.07  |
| user 2  | 0.93  | -0.21 | 0.07  |
| user 3  | 0.9   | -0.08 | 0.08  |
| user 4  | 0.93  | -0.08 | 0.08  |
| user 5  | 0.11  | 0.81  | -0.51 |
| user 6  | 0.12  | 0.81  | -0.51 |
| user 7  | 0.11  | 0.74  | -0.58 |
| user 8  | 0.02  | 0.74  | -0.51 |
| user 9  | -0.07 | 0.49  | 0.77  |
| user 10 | -0.07 | 0.49  | 0.77  |
| user 11 | -0.17 | 0.49  | 0.78  |
| user 12 | -0.17 | 0.49  | 0.78  |

X

|    | 红海行动 | 战狼2   | 湄公河行动2 | 汪汪队立大功 | 小猪佩奇  | 超级飞侠  | 小时代4 | 致青春   | 同桌的你  |
|----|------|-------|--------|--------|-------|-------|------|-------|-------|
| 动作 | 0.99 | 0.99  | 0.89   | 0.23   | 0.03  | 0.03  | 0.08 | -0.11 | -0.11 |
| 动画 | 0.09 | -0.14 | -0.14  | 0.87   | 0.58  | 0.58  | 0.23 | 0.36  | 0.36  |
| 爱情 | 0.16 | 0.07  | 0.07   | -0.41  | -0.47 | -0.47 | 0.82 | 0.74  | 0.74  |

# 什么是矩阵分解

- 某个用户u对电影i的预测评分 = User向量和Item向量的内积
- 把这两个矩阵相乘，就能得到每个用户对每部电影的预测评分了，评分值越大，表示用户喜欢该电影的可能性越大，该电影就越值得推荐给用户。

|         | 红海行动 | 战狼2   | 湄公河行动2 | 汪汪队立大功 | 小猪佩奇  | 超级飞侠  | 小时代4  | 致青春   | 同桌的你  |
|---------|------|-------|--------|--------|-------|-------|-------|-------|-------|
| user 1  | 0.92 | 0.94  | 0.84   | 0.12   | -0.05 | -0.05 | 0.11  | -0.08 | -0.08 |
| user 2  | 0.91 | 0.96  | 0.86   | 0.00   | -0.13 | -0.13 | 0.08  | -0.13 | -0.13 |
| user 3  | 0.90 | 0.91  | 0.82   | 0.10   | -0.06 | -0.06 | 0.12  | -0.07 | -0.07 |
| user 4  | 0.93 | 0.94  | 0.84   | 0.11   | -0.06 | -0.06 | 0.12  | -0.07 | -0.07 |
| user 5  | 0.10 | -0.04 | -0.05  | 0.94   | 0.71  | 0.71  | -0.22 | -0.10 | -0.10 |
| user 6  | 0.11 | -0.03 | -0.04  | 0.94   | 0.71  | 0.71  | -0.22 | -0.10 | -0.10 |
| user 7  | 0.08 | -0.04 | -0.05  | 0.91   | 0.71  | 0.71  | -0.30 | -0.17 | -0.17 |
| user 8  | 0.05 | -0.12 | -0.12  | 0.86   | 0.67  | 0.67  | -0.25 | -0.11 | -0.11 |
| user 9  | 0.10 | -0.08 | -0.07  | 0.09   | -0.08 | -0.08 | 0.74  | 0.75  | 0.75  |
| user 10 | 0.10 | -0.08 | -0.08  | 0.10   | -0.08 | -0.08 | 0.74  | 0.75  | 0.75  |
| user 11 | 0.00 | -0.18 | -0.17  | 0.07   | -0.09 | -0.09 | 0.74  | 0.77  | 0.77  |
| user 12 | 0.00 | -0.18 | -0.17  | 0.07   | -0.09 | -0.09 | 0.74  | 0.77  | 0.77  |

# 矩阵分解的目标函数

$r_{ui}$  表示用户u对item i 的评分

当  $>0$  时，表示有评分，当  $=0$  时，表示没有评分，

$x_u$  表示用户u的向量，k维列向量

$y_i$  表示item i 的向量，k维列向量

用户矩阵X，用户数为N

$$X = [x_1, x_2, \dots, x_N]$$

商品矩阵Y，商品数为M

$$Y = [y_1, y_2, \dots, y_M]$$

用户向量与物品向量的内积，表示用户u对物品i的预测评分

矩阵的目标函数：

$$\min_{X,Y} \sum_{r_{ui} \neq 0} (r_{ui} - x_u^T y_i)^2 + \lambda \left( \sum_u \|x_u\|_2^2 + \sum_i \|y_i\|_2^2 \right)$$

实际评分

L2正则项，保证数值计算稳定性，防止过拟合



# 矩阵分解的目标函数

---

目标函数最优化问题的工程解法：

- ALS, Alternating Least Squares, 交替最小二乘法
- SGD, Stochastic Gradient Descent, 随机梯度下降

# ALS求解方法

---

ALS, Alternative Least Square, ALS, 交替最小二乘法

- Step1, 固定Y 优化X
- Step2, 固定X 优化Y
- 重复Step1和2, 直到X 和Y 收敛。每次固定一个矩阵, 优化另一个矩阵, 都是最小二乘问题

# ALS求解方法

ALS, Alternative Least Square, ALS, 交替最小二乘法

- Step1, 固定Y优化X

$$\min_{X_u} \sum_{r_{ui} \neq 0} (r_{ui} - x_u^T y_i)^2 + \lambda \sum_u \|x_u\|_2^2$$

将目标函数转化为矩阵表达形式

$$J(x_u) = (R_u - Y_u^T x_u)^T (R_u - Y_u^T x_u) + \lambda x_u^T x_u$$



$$R_u = [r_{ui_1}, \dots, r_{ui_m}]^T$$

用户u 对m个物品的评分



$$Y_u = [y_{i_1}, \dots, y_{i_m}]$$

m 个物品的向量

# ALS求解方法

---

ALS, Alternative Least Square, ALS, 交替最小二乘法

- 对目标函数 $J$ 关于 $x_u$ 求梯度，并令梯度为零，得

$$\frac{\partial J(x_u)}{\partial x_u} = -2Y_u(R_u - Y_u^T x_u) + 2\lambda x_u = 0$$

求解后可得：

$$x_u = (Y_u Y_u^T + \lambda I)^{-1} Y_u R_u$$

# 奇异值分解SVD

我们可以得到为奇异值分解

$$A = P\Lambda Q^T$$

P为左奇异矩阵， $m \times m$ 维

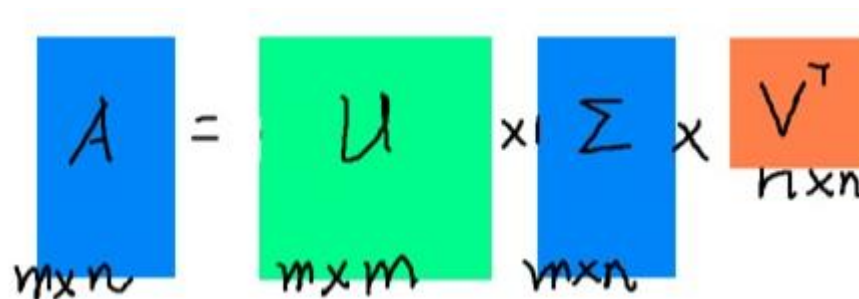
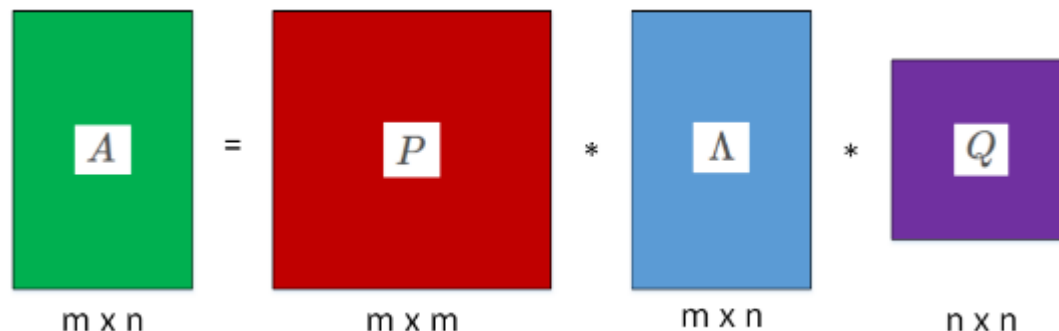
Q为右奇异矩阵， $n \times n$ 维

$\Lambda$ 对角线上的非零元素为特征值 $\lambda_1, \lambda_2, \dots, \lambda_k$

在推荐系统中

左奇异矩阵：User矩阵

右奇异矩阵：Item矩阵



# 奇异值分解SVD

比如,  $A = \begin{bmatrix} 1 & 2 \\ 1 & 1 \\ 0 & 0 \end{bmatrix}$

$AA^T = \begin{bmatrix} 5 & 3 & 0 \\ 3 & 2 & 0 \\ 0 & 0 & 0 \end{bmatrix}$

特征向量  $P = \begin{bmatrix} 0.85065081 & -0.52573111 & 0 \\ 0.52573111 & 0.85065081 & 0 \\ 0 & 0 & 1 \end{bmatrix}$

特征值  $\sigma_1 = 6.85410197, \sigma_2 = 0.14589803, \sigma_3 = 0$

$A^T A = \begin{bmatrix} 2 & 3 \\ 3 & 5 \end{bmatrix}$

特征向量  $Q = \begin{bmatrix} -0.52573111 & -0.85065081 \\ -0.85065081 & 0.52573111 \end{bmatrix}$

特征值  $\sigma_1 = 6.85410197, \sigma_2 = 0.14589803$

代入求得:

$$\lambda_1 = \sqrt{\sigma_1} = 2.61803399 \quad \lambda_2 = \sqrt{\sigma_2} = 0.38196601$$

$$P \Lambda Q^T = \begin{bmatrix} 0.85065081 & -0.52573111 & 0 \\ 0.52573111 & 0.85065081 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \lambda_1 & 0 \\ 0 & \lambda_2 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} -0.52573111 & -0.85065081 \\ -0.85065081 & 0.52573111 \end{bmatrix} = \begin{bmatrix} 1 & 2 \\ 1 & 1 \\ 0 & 0 \end{bmatrix}$$

# 奇异值分解SVD

---

$$P\Lambda Q^T = \begin{bmatrix} 0.85065081 & -0.52573111 & 0 \\ 0.52573111 & 0.85065081 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \lambda_1 & 0 \\ 0 & \lambda_2 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} -0.52573111 & -0.85065081 \\ -0.85065081 & 0.52573111 \end{bmatrix} = \begin{bmatrix} 1 & 2 \\ 1 & 1 \\ 0 & 0 \end{bmatrix}$$

奇异值分解：

$$A = \lambda_1 p_1 q_1^T + \lambda_2 p_2 q_2^T + \cdots + \lambda_k p_k q_k^T$$

$\lambda_1$ 为特征值， $p_1$ 为左奇异矩阵的特征向量

$q_1$ 为右奇异矩阵的特征向量

# 奇异值分解SVD

```
from scipy.linalg import svd
```

```
import numpy as np
```

```
from scipy.linalg import svd
```

```
A = np.array([[1,2],
```

```
              [1,1],
```

```
              [0,0]])
```

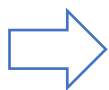
```
p,s,q = svd(A,full_matrices=False)
```

```
print('P=', p)
```

```
print('S=', s)
```

```
print('Q=', q)
```

$$P\Lambda Q^T = \begin{bmatrix} 0.85065081 & -0.52573111 & 0 \\ 0.52573111 & 0.85065081 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \lambda_1 & 0 \\ 0 & \lambda_2 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} -0.52573111 & -0.85065081 \\ -0.85065081 & 0.52573111 \end{bmatrix} = \begin{bmatrix} 1 & 2 \\ 1 & 1 \\ 0 & 0 \end{bmatrix}$$



```
P= [[-0.85065081 -0.52573111]
     [-0.52573111  0.85065081]
     [ 0.          0.          ]]
S= [2.61803399 0.38196601]
Q= [[-0.52573111 -0.85065081]
     [ 0.85065081 -0.52573111]]
[Finished in 0.6s]
```



# 奇异值分解SVD

---

如何理解SVD的作用

- 对于特征值矩阵，我们如果只包括某部分特征值，结果会怎样？

矩阵A：大小为1440\*1080的图片

- Step1，将图片转换为矩阵
- Step2，对矩阵进行奇异值分解，得到 $p, s, q$
- Step3，包括特征值矩阵中的K个最大特征值，其余特征值设置为0
- Step4，通过 $p, s', q$ 得到新的矩阵 $A'$ ，对比 $A'$ 与A的差别

# 奇异值分解SVD

---

如何理解矩阵分解

```
from scipy.linalg import svd
```

```
import matplotlib.pyplot as plt
```

```
# 取前k个特征，对图像进行还原
```

```
def get_image_feature(s, k):
```

```
    # 对于S，只保留前K个特征值
```

```
    s_temp = np.zeros(s.shape[0])
```

```
    s_temp[0:k] = s[0:k]
```

```
    s = s_temp * np.identity(s.shape[0])
```

```
    # 用新的s_temp，以及p,q重构A
```

```
    temp = np.dot(p,s)
```

```
    temp = np.dot(temp,q)
```

```
    plt.imshow(temp, cmap=plt.cm.gray, interpolation='nearest')
```

```
    plt.show()
```

```
# 加载256色图片
```

```
image = Image.open('./256.bmp')
```

```
A = np.array(image)
```

```
# 显示原图像
```

```
plt.imshow(A, cmap=plt.cm.gray, interpolation='nearest')
```

```
plt.show()
```

```
# 对图像矩阵A进行奇异值分解，得到p,s,q
```

```
p,s,q = svd(A, full_matrices=False)
```

```
# 取前k个特征，对图像进行还原
```

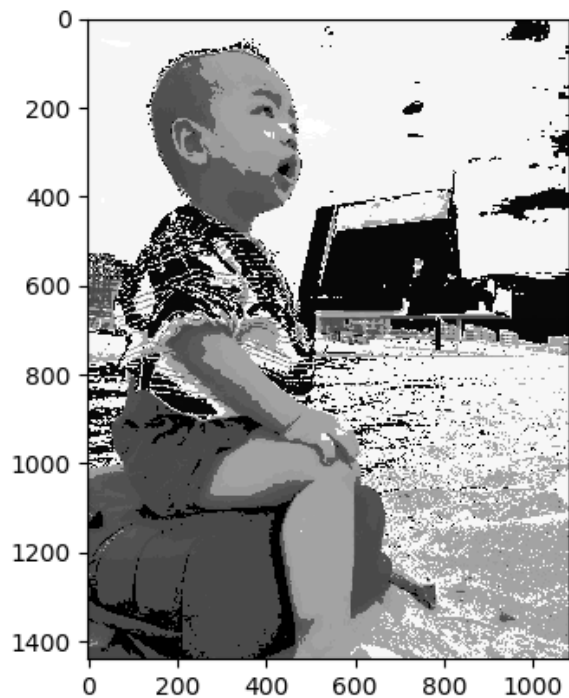
```
get_image_feature(s, 5)
```

```
get_image_feature(s, 50)
```

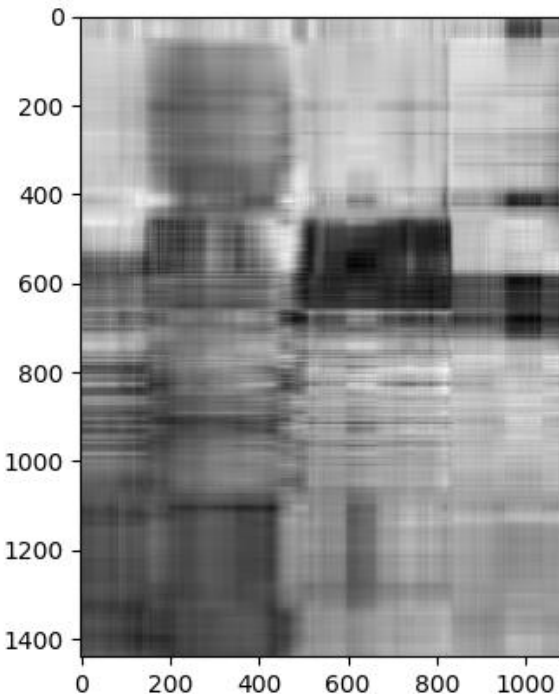
```
get_image_feature(s, 500)
```

# 奇异值分解SVD

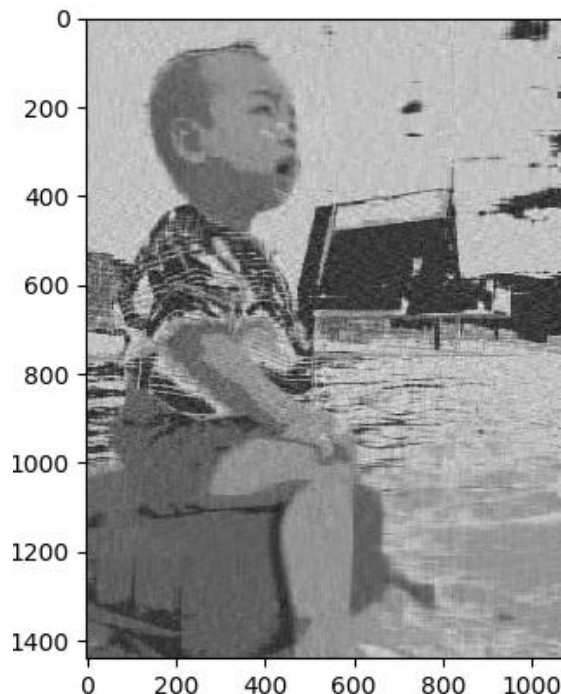
$$A = \lambda_1 p_1 q_1^T + \lambda_2 p_2 q_2^T + \cdots + \lambda_k p_k q_k^T$$



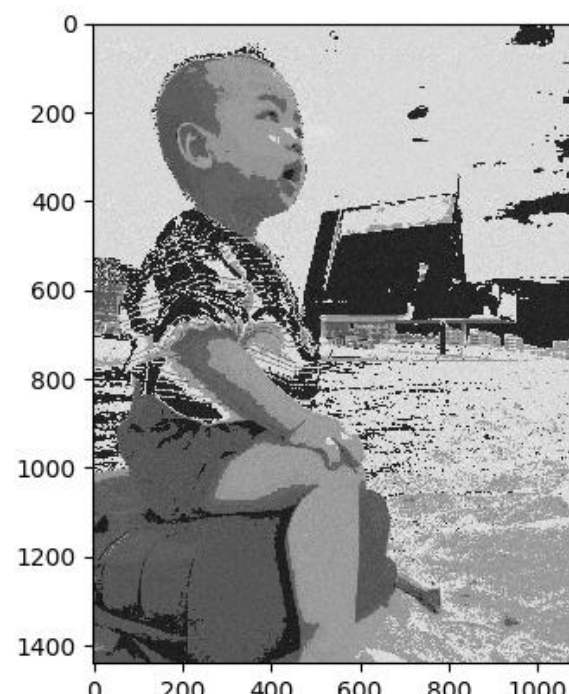
原始图片



k=5



k=50



k=500

少量的信息（比如10%），可以还原大部分图像信息（比如99%）

当K=50时，我们只需要保存  $(1440+1+1080)*50=126050$  个元素，占比  $126050/(1440*1080)=8\%$

# 传统SVD在推荐系统中的应用

将user-item评分问题，转化为SVD矩阵分解

$$\begin{matrix} \text{blue box} & = & \text{green box} & \times & \text{blue box} & \times & \text{orange box} \\ A & & U & & \Sigma & & V^T \\ m \times n & & m \times m & & m \times n & & n \times n \end{matrix}$$

$$\begin{bmatrix} 1 & 5 & 0 & 5 & 4 \\ 5 & 4 & 4 & 3 & 2 \\ 0 & 4 & 0 & 0 & 5 \\ 4 & 4 & 1 & 4 & 0 \\ 0 & 4 & 3 & 5 & 0 \\ 2 & 4 & 3 & 5 & 3 \end{bmatrix} = \begin{bmatrix} -0.46 & 0.40 & 0.30 & -0.43 & 0.32 & -0.50 \\ -0.46 & -0.30 & -0.65 & 0.28 & 0.02 & -0.44 \\ -0.25 & 0.75 & -0.28 & 0.16 & -0.46 & 0.22 \\ -0.38 & -0.35 & -0.13 & -0.68 & -0.32 & 0.38 \\ -0.38 & -0.24 & 0.62 & 0.38 & -0.50 & -0.13 \\ -0.48 & -0.01 & 0.10 & 0.31 & 0.57 & 0.59 \end{bmatrix} \begin{bmatrix} 16.47 & 0 & 0 & 0 & 0 \\ 0 & 6.21 & 0 & 0 & 0 \\ 0 & 0 & 4.40 & 0 & 0 \\ 0 & 0 & 0 & 2.90 & 0 \\ 0 & 0 & 0 & 0 & 1.58 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} -0.32 & -0.61 & -0.29 & -0.58 & -0.33 \\ -0.41 & 0.22 & -0.38 & -0.26 & 0.76 \\ -0.74 & 0.03 & -0.13 & 0.60 & -0.27 \\ -0.39 & -0.12 & 0.87 & -0.20 & 0.19 \\ 0.17 & -0.75 & -0.03 & 0.45 & 0.45 \end{bmatrix}$$

如果我们想要看user2对item3评分，可以得到

$$4 = -0.46 * 16.47 * (-0.29) + (-0.30) * 6.21 * (-0.38) + (-0.65) * 4.40 * (-0.13) + 0.28 * 2.90 * 0.87 + 0.02 * 1.58 * (-0.03)$$

- A中各元素=user行向量，item列向量，奇异值的加权内积

# 传统SVD在推荐系统中的应用

$$\begin{bmatrix} 1 & 5 & 0 & 5 & 4 \\ 5 & 4 & 4 & 3 & 2 \\ 0 & 4 & 0 & 0 & 5 \\ 4 & 4 & 1 & 4 & 0 \\ 0 & 4 & 3 & 5 & 0 \\ 2 & 4 & 3 & 5 & 3 \end{bmatrix} = \begin{bmatrix} -0.46 & 0.40 & 0.30 & -0.43 & 0.32 & -0.50 \\ -0.46 & -0.30 & -0.65 & 0.28 & 0.02 & -0.44 \\ -0.25 & 0.75 & -0.28 & 0.16 & -0.46 & 0.22 \\ -0.38 & -0.35 & -0.13 & -0.68 & -0.32 & 0.38 \\ -0.38 & -0.24 & 0.62 & 0.38 & -0.50 & -0.13 \\ -0.48 & -0.01 & 0.10 & 0.31 & 0.57 & 0.59 \end{bmatrix} \begin{bmatrix} 16.47 & 0 & 0 & 0 & 0 \\ 0 & 6.21 & 0 & 0 & 0 \\ 0 & 0 & 4.40 & 0 & 0 \\ 0 & 0 & 0 & 2.90 & 0 \\ 0 & 0 & 0 & 0 & 1.58 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} -0.32 & -0.61 & -0.29 & -0.58 & -0.33 \\ -0.41 & 0.22 & -0.38 & -0.26 & 0.76 \\ -0.74 & 0.03 & -0.13 & 0.60 & -0.27 \\ -0.39 & -0.12 & 0.87 & -0.20 & 0.19 \\ 0.17 & -0.75 & -0.03 & 0.45 & 0.45 \end{bmatrix}$$

实际上，我们发现user矩阵的最后一列是没有用到的，而且我们还可以使用更少的特征，比如特征个数=2

得到近似解A'

$$\begin{bmatrix} -0.46 & 0.40 \\ -0.46 & -0.30 \\ -0.25 & 0.75 \\ -0.38 & -0.35 \\ -0.38 & -0.24 \\ -0.48 & -0.01 \end{bmatrix} \begin{bmatrix} 16.47 & 0 \\ 0 & 6.21 \end{bmatrix} \begin{bmatrix} -0.32 & -0.61 & -0.29 & -0.58 & -0.33 \\ -0.41 & 0.22 & -0.38 & -0.26 & 0.76 \end{bmatrix} = \begin{bmatrix} 1.41 & 5.18 & 1.27 & 3.73 & 4.37 \\ 3.20 & 4.22 & 2.92 & 4.85 & 1.05 \\ -0.61 & 3.55 & -0.57 & 1.16 & 4.91 \\ 2.89 & 3.39 & 2.64 & 4.18 & 0.45 \\ 2.59 & 3.45 & 2.37 & 3.95 & 0.89 \\ 2.52 & 4.77 & 2.29 & 4.51 & 2.54 \end{bmatrix}$$

# 矩阵分解的应用场景

此外，在很多场景中也可以使用矩阵分解，比如 金融领域中的：

| 应用场景 | 分解对象    | 核心收益         | 关键技术       |
|------|---------|--------------|------------|
| 资产定价 | 收益-时间矩阵 | 分离系统风险与特有风险  | PCA、因子分析   |
| 风险分析 | 协方差矩阵   | 稳定估计、降维      | 谱分解、随机矩阵理论 |
| 组合优化 | 资产相关性   | 降低估计误差、稳定权重  | 主成分回归、因子模型 |
| 信用评分 | 用户-行为矩阵 | 自动特征学习、处理稀疏性 | 矩阵分解、协同过滤  |

# 金融市场中的定价模型

---

Thinking: 传统因子模型和 SVD 有什么区别?

从 Fama-French 到 SVD: 因子模型的两思路

思路一: 人工定义因子 (自上而下)

Fama-French 三因子模型是经典的资产定价框架:

股票收益 =  $\beta_{\text{市场}}$  x 市场溢价

+  $\beta_{\text{规模}}$  x 规模因子

+  $\beta_{\text{价值}}$  x 价值因子

+ 残差(个股特有)

先假设 市场、规模、价值 是驱动因子，再去数据中验证。

# 金融市场中的定价模型

思路二：让数据说话（自下而上）

SVD 的思路完全不同：

收益率矩阵  $R = U * \text{Sigma} * V^T$

- 不预设任何因子，直接对收益率矩阵做数学分解
- Sigma 告诉你：有几个因子在起作用
- U 告诉你：每只股票受这些因子影响多大
- $V^T$  告诉你：每个因子每天的表现

| 对比维度 | Fama-French    | SVD 分解       |
|------|----------------|--------------|
| 因子来源 | 人工假设（市场/规模/价值） | 数据自动发现       |
| 因子数量 | 预设 3-5 个       | 数据决定（看奇异值衰减） |
| 可解释性 | 强（每个因子有明确含义）   | 弱（需要事后溯源解读）  |
| 灵活性  | 低（结构固定）        | 高（随数据变化）     |
| 适用场景 | 学术研究、风险归因      | 降维、因子挖掘、市场监控 |



# 场景：收益率矩阵 SVD 分解

## 场景：收益率矩阵 SVD 分解

选取覆盖 6 个行业的 16 只股票，构建收益率矩阵 R:

R 的形状: 16只股票 x 722个交易日

|       | 日期1   | 日期2   | 日期3   | ... | 日期722 |
|-------|-------|-------|-------|-----|-------|
| 茅台    | +0.3% | -0.5% | +1.2% | ... | -0.9% |
| 五粮液   | +0.2% | -0.3% | +0.8% | ... | -0.7% |
| 平安银行  | -0.1% | +0.4% | +0.2% | ... | +0.3% |
| ...   |       |       |       |     |       |
| 纳指ETF | +0.5% | -0.8% | +1.5% | ... | +0.2% |



进行SVD 分解与奇异值衰减

SVD:  $R = U * \text{Sigma} * V^T$

- U (16 x 16): 每只股票对隐因子的暴露（因子loading）
- Sigma (16) : 奇异值，衡量每个隐因子的重要程度
- $V^T$  (16 x 722): 每个隐因子在每天的表现

# 场景：收益率矩阵 SVD 分解

收益率矩阵的构造：

将 16 只股票 722 个交易日的日收益率组织成矩阵  $R$ ，形状为  $16 \times 722$ （股票数  $\times$  交易日数）。

每一行代表一只股票的收益率时间序列，每一列代表某一天所有股票的截面收益率。

**Thinking:** 为什么要中心化？

对每只股票减去其均值，消除各股票长期收益差异的干扰，让 SVD 专注于捕捉协方差结构（即股票之间的联动模式）。

```
# 构建收益率矩阵并执行SVD分解
```

```
R = returns_df.values.T # (N, T) = (16, 722)
```

```
R_centered = R - R.mean(axis=1, keepdims=True)
```

```
# 核心: 奇异值分解
```

```
U, sigma, Vt = np.linalg.svd(R_centered, full_matrices=False)
```

```
# 方差解释率计算
```

```
total_var = np.sum(sigma ** 2)
```

```
explained_ratio = sigma ** 2 / total_var
```

```
cumulative_ratio = np.cumsum(explained_ratio)
```

```
print(f"SVD分解完成: R_centered ({R_centered.shape[0]} x  
{R_centered.shape[1]})")
```

```
print(f"奇异值个数: {len(sigma)}")
```

# 场景：收益率矩阵 SVD 分解

奇异值衰减分析结果:

| 隐因子 | 方差占比   | 累积占比   | 解读          |
|-----|--------|--------|-------------|
| #1  | 40.70% | 40.70% | 市场因子 (beta) |
| #2  | 16.80% | 57.50% | 行业/板块轮动     |
| #3  | 10.30% | 67.80% | 动量/反转       |
| #4  | 5.70%  | 73.50% | 波动率/规模      |
| #5  | 5.00%  | 78.50% | 其他结构因子      |
| #6  | 4.40%  | 82.90% | ...         |



16 只股票的收益率，主要由 6 个隐因子驱动（累积方差 80%），前 3 个因子就解释了 67.8% 的总方差。

关键发现:

第一个因子独占 40.7%，说明 A 股市场齐涨齐跌的特征非常显著  
=> 这与华泰研究中 市场因子(beta)是最大风险来源的结论一致。

# 场景：收益率矩阵 SVD 分解

---

Thinking: 什么是方差解释率？

在 SVD 语境下，方差不是统计学里单个变量的方差，而是整个收益率矩阵的总波动能量。

方差占比  $k = \sigma_k^2 / (\sigma_1^2 + \sigma_2^2 + \dots + \sigma_N^2)$

$\sigma_k$  是第  $k$  个奇异值

$\sigma_k^2$  代表第  $k$  个隐因子所解释的波动能量

方差占比衡量该因子在整体市场波动中的贡献权重

# 场景：收益率矩阵 SVD 分解

Thinking: 40.71% 是怎么算出来的？

SVD 分解得到 16 个奇异值（因为有 16 只股票），其中前 4 个为：

| 因子  | 奇异值 $\sigma_k$ | $\sigma_k^2$ (因子能量) | 含义          |
|-----|----------------|---------------------|-------------|
| #1  | 3.8542         | $3.8542^2 = 14.855$ | 第一因子捕获的波动能量 |
| #2  | 2.4736         | $2.4736^2 = 6.119$  | 第二因子捕获的波动能量 |
| #3  | 1.9423         | $1.9423^2 = 3.773$  | 第三因子捕获的波动能量 |
| #4  | 1.5821         | $1.5821^2 = 2.503$  | 第四因子捕获的波动能量 |
| ... | ...            | ...                 | (共 16 个因子)  |

16 个因子的能量之和（总方差）：

总方差 =  $\sigma_1^2 + \sigma_2^2 + \dots + \sigma_{16}^2 = 14.855 + 6.119 + 3.773 + 2.503 + \dots = 36.49$

第一因子占比 =  $\sigma_1^2 / \text{总方差} = 14.855 / 36.49 = 40.71\%$

想象 16 只股票每天的涨跌幅是一团总能量。SVD 把这团能量分解成 16 个独立的驱动力。第一个驱动力（因子 #1）一个人就吃掉了 40.71% 的能量  
  
=> 说明市场中有一个压倒性的共同力量在同时推动所有股票。

# 场景：收益率矩阵 SVD 分解

Thinking: 怎么看出 这是市场因子，这是行业轮动？

SVD 分解出的因子是纯数学对象，没有自带标签。市场因子，行业轮动 这些名字是我们通过三步分析法推断出来

分析工具一：U 矩阵 -- 看谁在一起动

U 矩阵的第 k 列 U[:, k] 告诉我们：每只股票对第 k 个因子的载荷（暴露程度）。

| 股票    | 行业   | Factor1 载荷 | Factor2 载荷 | Factor3 载荷 | Factor4 载荷 |
|-------|------|------------|------------|------------|------------|
| 贵州茅台  | 食品饮料 | +0.28      | +0.35      | -0.12      | +0.08      |
| 五粮液   | 食品饮料 | +0.26      | +0.33      | -0.09      | +0.06      |
| 洋河股份  | 食品饮料 | +0.25      | +0.30      | -0.11      | +0.05      |
| 宁德时代  | 新能源  | +0.32      | -0.28      | +0.25      | +0.30      |
| 隆基绿能  | 新能源  | +0.30      | -0.31      | +0.28      | +0.32      |
| 中芯国际  | 科技   | +0.31      | -0.26      | -0.20      | +0.15      |
| 北方华创  | 科技   | +0.29      | -0.24      | -0.22      | +0.18      |
| 工商银行  | 银行   | +0.15      | +0.18      | +0.05      | -0.35      |
| 招商银行  | 银行   | +0.22      | +0.15      | +0.08      | -0.28      |
| 纳指ETF | ETF  | +0.05      | -0.02      | +0.03      | +0.02      |

表中数值为示意，实际运行代码  
可得到精确值。  
绿色=正载荷，红色=负载荷

# 场景：收益率矩阵 SVD 分解

Thinking: Factor 1 = 市场因子（Beta）-- 怎么看出来的？

判断依据一：U 矩阵第一列全同号

观察上表 Factor1 载荷列：几乎所有 A 股股票的载荷都是正数（+0.15 ~ +0.32），只有纳指 ETF 接近零。

| 股票    | 行业   | Factor1 载荷 | Factor2 载荷 | Factor3 载荷 | Factor4 载荷 |
|-------|------|------------|------------|------------|------------|
| 贵州茅台  | 食品饮料 | +0.28      | +0.35      | -0.12      | +0.08      |
| 五粮液   | 食品饮料 | +0.26      | +0.33      | -0.09      | +0.06      |
| 洋河股份  | 食品饮料 | +0.25      | +0.30      | -0.11      | +0.05      |
| 宁德时代  | 新能源  | +0.32      | -0.28      | +0.25      | +0.30      |
| 隆基绿能  | 新能源  | +0.30      | -0.31      | +0.28      | +0.32      |
| 中芯国际  | 科技   | +0.31      | -0.26      | -0.20      | +0.15      |
| 北方华创  | 科技   | +0.29      | -0.24      | -0.22      | +0.18      |
| 工商银行  | 银行   | +0.15      | +0.18      | +0.05      | -0.35      |
| 招商银行  | 银行   | +0.22      | +0.15      | +0.08      | -0.28      |
| 纳指ETF | ETF  | +0.05      | -0.02      | +0.03      | +0.02      |

当 Factor1 上升 1 个单位时，所有股票都一起涨（只是幅度不同）。  
当 Factor1 下降 1 个单位时，所有股票都一起跌。  
=> 这就是齐涨齐跌 -- 市场 Beta 的典型特征。

# 场景：收益率矩阵 SVD 分解

判断依据二：与等权大盘高度相关

代码 step8 中计算了 Factor1 时间序列与 16 只股票等权收益率 的相关性：

$\text{corr}(\text{Factor1}, \text{等权大盘}) = +0.85 \sim +0.95$ （高度正相关）

| 股票    | 行业   | Factor1 载荷 | Factor2 载荷 | Factor3 载荷 | Factor4 载荷 |
|-------|------|------------|------------|------------|------------|
| 贵州茅台  | 食品饮料 | +0.28      | +0.35      | -0.12      | +0.08      |
| 五粮液   | 食品饮料 | +0.26      | +0.33      | -0.09      | +0.06      |
| 洋河股份  | 食品饮料 | +0.25      | +0.30      | -0.11      | +0.05      |
| 宁德时代  | 新能源  | +0.32      | -0.28      | +0.25      | +0.30      |
| 隆基绿能  | 新能源  | +0.30      | -0.31      | +0.28      | +0.32      |
| 中芯国际  | 科技   | +0.31      | -0.26      | -0.20      | +0.15      |
| 北方华创  | 科技   | +0.29      | -0.24      | -0.22      | +0.18      |
| 工商银行  | 银行   | +0.15      | +0.18      | +0.05      | -0.35      |
| 招商银行  | 银行   | +0.22      | +0.15      | +0.08      | -0.28      |
| 纳指ETF | ETF  | +0.05      | -0.02      | +0.03      | +0.02      |

如果 Factor1 是行业因子，它应该和某些行业正相关、另一些负相关；但它和几乎所有行业都正相关 => 所以它不是行业因子，而是整体市场涨跌的共同方向。



# 场景：收益率矩阵 SVD 分解

判断依据三：解释力最大（40.71%）

金融学文献中，对任何股票组合做 PCA/SVD，第一主成分几乎总是市场因子，因为：

- A 股散户占比高，情绪传染导致齐涨齐跌
- 央行政策、宏观经济等系统性因素同时影响所有股票
- 华泰金工研究表明，A 股 Beta 因子单因子解释方差通常在 35%-45%

| 股票    | 行业   | Factor1 载荷 | Factor2 载荷 | Factor3 载荷 | Factor4 载荷 |
|-------|------|------------|------------|------------|------------|
| 贵州茅台  | 食品饮料 | +0.28      | +0.35      | -0.12      | +0.08      |
| 五粮液   | 食品饮料 | +0.26      | +0.33      | -0.09      | +0.06      |
| 洋河股份  | 食品饮料 | +0.25      | +0.30      | -0.11      | +0.05      |
| 宁德时代  | 新能源  | +0.32      | -0.28      | +0.25      | +0.30      |
| 隆基绿能  | 新能源  | +0.30      | -0.31      | +0.28      | +0.32      |
| 中芯国际  | 科技   | +0.31      | -0.26      | -0.20      | +0.15      |
| 北方华创  | 科技   | +0.29      | -0.24      | -0.22      | +0.18      |
| 工商银行  | 银行   | +0.15      | +0.18      | +0.05      | -0.35      |
| 招商银行  | 银行   | +0.22      | +0.15      | +0.08      | -0.28      |
| 纳指ETF | ETF  | +0.05      | -0.02      | +0.03      | +0.02      |

不是因为它叫Factor1就自动是市场因子。是因为观察到：

(1) 所有股票载荷同号；

(2) 与大盘收益率高度相关；

(3) 占比符合金融学先验认知

=> 市场因子。

# 场景：收益率矩阵 SVD 分解

Thinking: Factor 2 = 行业/板块轮动 -- 怎么看出来的？

关键证据：U 矩阵第二列行业间反号（观察 Factor2 载荷列，出现了明显的行业分化）：

食品饮料、银行：载荷为正（+0.15 ~ +0.35）-- 防御型板块

新能源、科技：载荷为负（-0.24 ~ -0.31）-- 进攻型板块

| 股票    | 行业   | Factor1 载荷 | Factor2 载荷 | Factor3 载荷 | Factor4 载荷 |
|-------|------|------------|------------|------------|------------|
| 贵州茅台  | 食品饮料 | +0.28      | +0.35      | -0.12      | +0.08      |
| 五粮液   | 食品饮料 | +0.26      | +0.33      | -0.09      | +0.06      |
| 洋河股份  | 食品饮料 | +0.25      | +0.30      | -0.11      | +0.05      |
| 宁德时代  | 新能源  | +0.32      | -0.28      | +0.25      | +0.30      |
| 隆基绿能  | 新能源  | +0.30      | -0.31      | +0.28      | +0.32      |
| 中芯国际  | 科技   | +0.31      | -0.26      | -0.20      | +0.15      |
| 北方华创  | 科技   | +0.29      | -0.24      | -0.22      | +0.18      |
| 工商银行  | 银行   | +0.15      | +0.18      | +0.05      | -0.35      |
| 招商银行  | 银行   | +0.22      | +0.15      | +0.08      | -0.28      |
| 纳指ETF | ETF  | +0.05      | -0.02      | +0.03      | +0.02      |

当 Factor2 上升时，食品饮料涨、新能源跌；

当 Factor2 下降时，食品饮料跌、新能源涨。这就是典型的板块轮动

=> 资金从一类行业流向另一类行业。

# 场景：收益率矩阵 SVD 分解

验证方法：计算 Factor2 时间序列与各行业等权收益率的相关性，应该看到：

| Factor2 与行业相关性 | 食品饮料 | 银行   | 新能源  | 科技   | 医药   |
|----------------|------|------|------|------|------|
| 相关系数           | +0.6 | +0.3 | -0.5 | -0.4 | +0.1 |

正相关与负相关的行业泾渭分明，验证了板块轮动的判断。

# 场景：收益率矩阵 SVD 分解

---

Thinking: Factor 3 = 动量/反转 -- 怎么看出来的？

结合时间序列特征：

方法一：看 U 矩阵中 高波动 vs 低波动 的分化

新能源（宁德时代 +0.25、隆基绿能 +0.28）：近期涨幅大的股票，载荷为正

科技（中芯国际 -0.20、北方华创 -0.22）：近期跌幅大的股票，载荷为负

两组恰好是 前期涨的继续涨，前期跌的继续跌 或相反 -- 对应动量或反转

# 场景：收益率矩阵 SVD 分解

---

方法二：看 VT 时间序列的自相关性

将 Factor3 的每日收益做自相关分析：

- 如果短期正自相关（20 日内）-- 动量效应（趋势延续）
- 如果短期负自相关 -- 反转效应（均值回归）
- 实际上 A 股短周期以反转为主、中长周期以动量为主，所以该因子命名为 动量/反转

# 场景：收益率矩阵 SVD 分解

---

方法三：与已知动量因子做相关性检验

计算 Factor3 与手工构造的 20 日动量因子（过去 20 日收益率排名前 50% vs 后 50%）的相关性

=> 如果显著相关，则确认其动量含义。

# 场景：收益率矩阵 SVD 分解

---

Thinking: Factor 4 = 波动率/规模 -- 怎么看出来的？

关键洞察：载荷与股票特征（波动率、市值）相关

观察 Factor4 载荷：

- 正载荷：新能源（宁德时代 +0.30、隆基绿能 +0.32）-- 高波动、中市值
- 负载荷：银行（工商银行 -0.35、招商银行 -0.28）-- 低波动、大市值

Factor4 区分的是 高波动的中小市值成长股 vs 低波动的大市值价值股。

当 Factor4 为正，成长股跑赢价值股；为负则反之 => 这就是经典的波动率/规模因子。

验证方法：计算每只股票的 Factor4 载荷与其历史波动率的相关性。

如果  $\text{corr} > 0.6$ ，确认这是波动率因子。

# Summary

| 步骤         | 操作                      | 观察什么            | 判断逻辑                                   |
|------------|-------------------------|-----------------|----------------------------------------|
| 第一步：看 U 矩阵 | 查看第 k 列所有股票的载荷          | 载荷的正负和大小        | 全同号 = 市场因子；行业间反号 = 行业轮动；与某特征相关 = 该特征因子 |
| 第二步：看相关性   | 将 $V^T$ 时间序列与已知指标做 corr | 与大盘/行业/动量等的相关系数 | 高相关确认因子含义，低相关排除候选                      |
| 第三步：看时间序列  | 观察 $V^T$ 的累积净值走势        | 走势是否与已知市场事件吻合   | 如：924 行情期间 Factor1 飙升 = 确认是市场因子        |

Thinking：每个因子只对应一个经济含义？

不一定。Factor3 可能同时包含动量和反转成分，因此命名为"动量/反转"



# 场景：滚动 SVD -- 市场状态监控

## 场景：滚动 SVD -- 市场状态监控

核心指标：第一因子方差占比

固定窗口 SVD 假设因子结构不变，但现实中：

- 齐涨齐跌期: Factor1 占比高 (>50%)，beta 主导
- 板块分化期: Factor1 占比中等 (35%-50%)，行业轮动
- 个股行情期: Factor1 占比低 (<30%)，alpha 机会多



## 实测结果（120 天滚动窗口）

| 时间              | Factor1 占比 | 市场状态       |
|-----------------|------------|------------|
| 2023年初          | ~28%       | 个股行情       |
| 2023年底          | ~45%       | 板块分化       |
| 2024年10月(924行情) | ~57%       | 齐涨齐跌 (最高!) |
| 2025年下半年        | ~33%       | 个股行情       |

## 场景：滚动 SVD -- 市场状态监控

```
window = 120 # 约半年
step = 20    # 约一个月
results = []

for start in range(0, T - window, step):
    end = start + window
    window_data = returns_df.iloc[start:end]
    R_w = window_data.values.T
    R_w = R_w - R_w.mean(axis=1, keepdims=True)
    _, sigma_w, _ = np.linalg.svd(R_w, full_matrices=False)
    top1_ratio = sigma_w[0] ** 2 / np.sum(sigma_w ** 2)
    top3_ratio = np.sum(sigma_w[:3] ** 2) / np.sum(sigma_w ** 2)
```

```
# 判断市场状态
if top1_ratio > 0.50:
    regime = "齐涨齐跌"
elif top1_ratio > 0.30:
    regime = "板块分化"
else:
    regime = "个股行情"
results.append((date_label, top1_ratio, top3_ratio, regime))
```

# 场景：滚动 SVD -- 市场状态监控

滚动窗口: 120天, 步长: 20天

第一因子方差占比变化:

| 时间段     | Factor1占比 | Top3占比 | 市场状态  |
|---------|-----------|--------|-------|
| -----   | -----     | -----  | ----- |
| 2023-07 | 28.3%     | 58.2%  | 个股行情  |
| 2023-09 | 31.5%     | 62.1%  | 个股行情  |
| 2023-11 | 38.2%     | 68.5%  | 板块分化  |
| 2024-01 | 42.1%     | 72.3%  | 板块分化  |
| 2024-03 | 35.6%     | 66.8%  | 板块分化  |
| 2024-05 | 33.2%     | 64.1%  | 板块分化  |
| 2024-07 | 36.8%     | 67.5%  | 板块分化  |
| 2024-09 | 45.2%     | 74.8%  | 板块分化  |
| 2024-10 | 57.3%     | 82.1%  | 齐涨齐跌  |
| 2024-12 | 48.5%     | 76.2%  | 板块分化  |
| 2025-02 | 39.1%     | 69.3%  | 板块分化  |

|         |       |       |      |
|---------|-------|-------|------|
| 2025-04 | 35.2% | 65.8% | 板块分化 |
| 2025-06 | 33.8% | 63.5% | 板块分化 |
| 2025-08 | 31.2% | 61.8% | 个股行情 |

统计:

Factor1平均占比: 37.3%  
最高(齐涨齐跌): 57.3% (2024-10) <-- 924行情!  
最低(个股行情): 28.3% (2023-07)

2024年10月 Factor1 占比飙升至 57.3%，恰好对应 924行情后的普涨行情。当时政策组合拳密集出台，市场短期内呈现高度同步的齐涨齐跌特征。  
=> SVD 滚动监控成功捕捉到了这一状态转换。

# 场景：残差分析 -- 发现有 Alpha 的个股

场景：残差分析 -- 发现有 Alpha 的个股

残差 = 无法被隐因子解释的部分

$R = U * \text{Sigma} * V^T + e$

- $R_{\text{approx}}$  = 用前5个隐因子重构的收益
- 残差  $e = R - R_{\text{approx}}$  = 个股特有风险



残差大: 股票有自己独特的驱动力（个股 alpha）

残差小: 完全被市场/行业因子驱动（纯 beta）

| 股票     | 行业  | 残差占比   | 解读                   |
|--------|-----|--------|----------------------|
| 纳指 ETF | ETF | 98.70% | A 股因子完全解释不了（当然，它是美股） |
| 工商银行   | 银行  | 63.40% | 独立行情强（国有大行有自己的逻辑）    |
| 恒瑞医药   | 医药  | 35.90% | 有 alpha（新药研发等个股逻辑）   |
| 中芯国际   | 科技  | 1.80%  | 纯 beta（完全被科技因子解释）    |
| 晶澳科技   | 新能源 | 4.40%  | 纯 beta（完全跟随新能源板块）    |

## 场景：残差分析 -- 发现有 Alpha 的个股

```
# 用前5个隐因子重构收益率矩阵
n_factors_reconstruct = 5
R_approx = U[:, :n_factors_reconstruct] @ \
    np.diag(sigma[:n_factors_reconstruct]) @ \
    Vt[:n_factors_reconstruct, :]

# 计算残差
residual = R_centered - R_approx

# 残差占比 = 残差方差 / 原始方差
residual_var = np.var(residual, axis=1)
original_var = np.var(R_centered, axis=1)
residual_ratio = residual_var / original_var
residual_std = np.std(residual, axis=1)
```

# 场景：残差分析 -- 发现有 Alpha 的个股

使用前5个隐因子重构, 各股残差占比:

| 代码        | 名称    | 行业  | 残差占比  | 残差波动   | 解读       |
|-----------|-------|-----|-------|--------|----------|
| -----     |       |     |       |        |          |
| 159941.SZ | 纳指ETF | ETF | 98.7% | 0.0156 | 独立行情强    |
| 601398.SH | 工商银行  | 银行  | 63.4% | 0.0078 | 独立行情强    |
| 000538.SZ | 云南白药  | 医药  | 35.9% | 0.0062 | 有alpha信号 |
| 600276.SH | 恒瑞医药  | 医药  | 32.1% | 0.0058 | 有alpha信号 |
| 000001.SZ | 平安银行  | 银行  | 28.5% | 0.0054 | 跟随大盘     |
| 600036.SH | 招商银行  | 银行  | 25.3% | 0.0051 | 跟随大盘     |
| 600519.SH | 贵州茅台  | 食品  | 22.8% | 0.0048 | 跟随大盘     |
| 000858.SZ | 五粮液   | 食品  | 18.6% | 0.0043 | 跟随大盘     |
| 300760.SZ | 迈瑞医疗  | 医药  | 15.2% | 0.0039 | 跟随大盘     |
| 002304.SZ | 洋河股份  | 食品  | 12.5% | 0.0035 | 跟随大盘     |
| 300750.SZ | 宁德时代  | 新能源 | 10.8% | 0.0033 | 跟随大盘     |
| 601012.SH | 隆基绿能  | 新能源 | 8.5%  | 0.0031 | 跟随大盘     |
| 002371.SZ | 北方华创  | 科技  | 6.2%  | 0.0027 | 跟随大盘     |

|           |      |     |      |        |      |
|-----------|------|-----|------|--------|------|
| 688012.SH | 中微公司 | 科技  | 5.1% | 0.0025 | 跟随大盘 |
| 002459.SZ | 晶澳科技 | 新能源 | 4.4% | 0.0028 | 跟随大盘 |
| 688981.SH | 中芯国际 | 科技  | 1.8% | 0.0021 | 跟随大盘 |

投资含义:

高alpha个股: 纳指ETF, 工商银行, 云南白药

-> 这些股票走势独立, 适合做个股择时

纯beta个股: 中芯国际, 晶澳科技, 北方华创

-> 这些股票完全跟随市场/行业, 可用因子模型复制

# SVD因子降维

---

实际量化策略中，常用因子库可能包含 50+ 甚至上百个因子。

直接全部输入模型会导致：

- 多重共线性：许多因子高度相关（如 PE 和 PB 同属价值类）
- 维度灾难：样本量不够支撑高维特征空间
- 过拟合：模型记住了噪声而非信号

Thinking: 有什么解决方案么？

解决方案：SVD/PCA 压缩

用 SVD 将 52 维原始因子压缩为 K 个主成分，保留最重要的方差信息，丢弃噪声维度。

# SVD因子降维

```
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_scaled = scaler.fit_transform(factor_matrix)

results = {}
for k in [3, 5, 8, 10, 15]:
    pca = PCA(n_components=k)
    X_pca = pca.fit_transform(X_scaled)
    var_retained = pca.explained_variance_ratio_.sum()
    # 训练XGBoost并评估
    model = XGBClassifier(n_estimators=100, max_depth=4)
    model.fit(X_pca_train, y_train)

    auc = roc_auc_score(y_test, model.predict_proba(X_pca_test)[: , 1])
    results[k] = {'var': var_retained, 'auc': auc}
```

XGBoost预测效果对比:

| 方法       | 维度 | 方差保留   | AUC    | Accuracy |
|----------|----|--------|--------|----------|
| -----    |    |        |        |          |
| 全部因子     | 52 | 100.0% | 0.5423 | 0.5312   |
| SVD K=3  | 3  | 45.2%  | 0.5198 | 0.5156   |
| SVD K=5  | 5  | 62.8%  | 0.5356 | 0.5267   |
| SVD K=8  | 8  | 78.5%  | 0.5412 | 0.5298   |
| SVD K=10 | 10 | 85.3%  | 0.5401 | 0.5289   |
| SVD K=15 | 15 | 93.6%  | 0.5418 | 0.5308   |

结论:

SVD K=8 用 8 个主成分保留 78.5% 方差

AUC 从 0.5423 仅降至 0.5412 (差 0.0011)

-> 大部分因子信息是冗余的

选择 K 值时, 通常取累积方差达到 75%-85% 的点。  
=> K=8 是这里最佳平衡点 -- 维度从 52 降至 8 (降幅 85%), AUC 几乎无损失。更大的 K 值带来的边际收益已微乎其微。



# SVD因子降维

---

Thinking: SVD 因子到底是什么？

想象你有 52 种颜料（原始因子），但实际上只需要 8 种基础色（主成分）就能调出所有颜色。

=> SVD 做的就是找出这 8 种基础色，以及每种基础色的配方。

VT 矩阵就是这个配方表：它告诉我们每个主成分由哪些原始因子混合而成。

# SVD因子降维

## 因子载荷分析示例

| 主成分 | 主要构成因子      | 权重区间        | 直觉解释          |
|-----|-------------|-------------|---------------|
| PC1 | 市值、成交额、流通市值 | 0.35 ~ 0.42 | 规模因子 -- 大小盘分化 |
| PC2 | PE、PB、EP    | 0.30 ~ 0.38 | 价值因子 -- 估值高低  |
| PC3 | 20日动量、60日动量 | 0.28 ~ 0.35 | 动量因子 -- 趋势延续  |
| PC4 | 波动率、振幅、换手率  | 0.25 ~ 0.33 | 波动因子 -- 风险偏好  |
| PC5 | ROE、ROA、毛利率 | 0.22 ~ 0.30 | 质量因子 -- 盈利能力  |

原始因子的优势在于含义清晰（PE 就是市盈率），而 SVD 主成分的优势在于去除冗余、降低共线性。

=> 实际生产中，可以先用 SVD 压缩建模，再用原始因子做归因解释，两者互补。

# Summary

---

## SVD 在量化中的 3 大应用

### 应用一：隐因子发现

通过对收益率矩阵做 SVD 分解，无需先验假设即可发现市场的主要驱动力。

碎石图告诉我们市场由几个独立因子驱动，U 矩阵揭示每只股票在各因子上的暴露程度。

适用场景：研究阶段，探索新市场或新资产类别的因子结构

=> 不被已有因子框架局限

### 应用二：市场状态监控

通过滚动 SVD 监控第一因子方差占比，实时判断市场处于齐涨齐跌、板块分化还是个股行情阶段。

适用场景：策略配置层，动态调整策略权重

=> 齐涨齐跌期加仓 beta 策略，个股行情期切换 alpha 策略

# Summary

---

## 应用三：因子压缩降维

将高维因子空间压缩为低维主成分，既保留了主要信息，又消除了多重共线性和噪声。

适用场景：机器学习建模前的特征工程

=> 52 维压缩到 8 维，AUC 几乎无损失，训练速度大幅提升

# 打卡：SVD因子分析



针对你的股票池（也可以是全量的大A股票），进行SVD因子分析

- 通过对收益率矩阵做 SVD 分解，挖掘市场的主要驱动力。
- 市场状态监控

通过滚动 SVD 监控第一因子方差占比，判断市场处于哪种阶段  
齐涨齐跌、板块分化还是个股行情阶段。

你的自选股是走独立行情么？还是跟随大盘？

# CASE：实盘交易绩效分析

# CASE：实盘交易绩效分析

---

TO DO：针对你的证券交易，制作绩效报告

把券商导出的历史成交 CSV 解析成结构化数据，完成个股盈亏拆解、交易成本统计、按市价逐日估值的组合净值曲线，再用 QuantStats 生成绩效报告。

Step1，导出 券商历史成交CSV

Step2，绩效分析

盈亏/成本/NAV/QuantStats

Step3，SVD状态分析

# CASE：实盘交易绩效分析

登录 PC 版客户端

顶部导航：账户 → 普通交易 → 历史成交

选择起止日期（单次区间通常不超过约 90 天，可多次导出）

下载 CSV；多个文件放在同一工作目录，脚本可自动合并





# CASE：实盘交易绩效分析

< 返回

🏠 首页 / 普通交易 / 历史成交

起始日期

2026-01-01

终止日期

2026-03-25

查询

导出

筛选

汇总

资金账号:

| 序号 | 证券名称   | 证券代码   | 成交日期       | 成交时间     | 买卖方向 | 成交数量  | 成交价格   | 成交金额    | 佣金    | 净佣金   | 过户费  | 印花税     | 结算费 | 成交编号          | 股东代码       | 备注                                           |
|----|--------|--------|------------|----------|------|-------|--------|---------|-------|-------|------|---------|-----|---------------|------------|----------------------------------------------|
| 1  | 九安医疗   | 002432 | 2026-03-06 | 09:31:00 | 买入   | 3000  | 49.51  | 148530  | 22.28 | 14.24 | 0    | 0       | 0   | 4214363       | 0331641444 | 九安医疗证券买入                                     |
| 2  | 新天然气   | 603393 | 2026-03-04 | 14:20:00 | 卖出   | 2000  | 40.8   | 81600   | 12.24 | 7.83  | 0.81 | 40.8    | 0   | 66354820      | A484381710 | 新天然气证券卖出                                     |
| 3  | 联合能源集团 | 00467  | 2026-03-03 | --       | 卖出   | 50000 | 0.96   | 42249.6 | 25.35 | --    | 0    | 42.2496 | 0   | 467000006405  | A484381710 | 港股待交收卖出<br>成交00467价格<br>0.96港元数量<br>50000.00 |
| 4  | 联合能源集团 | 00467  | 2026-03-03 | --       | 买入   | 50000 | 0.93   | 40938.6 | 24.57 | --    | 0    | 41.3788 | 0   | 4670000005583 | A484381710 | 港股待交收买入<br>成交00467价格<br>0.93港元数量<br>50000.00 |
| 5  | 新天然气   | 603393 | 2026-03-03 | 10:19:00 | 买入   | 2000  | 40.85  | 81700   | 12.26 | 7.84  | 0.82 | 0       | 0   | 36544608      | A484381710 | 新天然气证券买入                                     |
| 6  | 贵州茅台   | 600519 | 2026-02-24 | 14:15:00 | 卖出   | 100   | 1466   | 146600  | 21.99 | 14.06 | 1.47 | 73.3    | 0   | 51331567      | A484381710 | 贵州茅台证券卖出                                     |
| 7  | 纳指ETF  | 513100 | 2026-01-20 | 14:55:00 | 买入   | 100   | 1.876  | 187.6   | 5     | 4.99  | 0    | 0       | 0   | 75717286      | A484381710 | 纳指ETF证券买入                                    |
| 8  | 贵州茅台   | 600519 | 2026-01-20 | 09:25:00 | 买入   | 100   | 1379.5 | 137950  | 18.68 | 11.22 | 1.38 | 0       | 0   | 302563        | A484381710 | 贵州茅台证券买入                                     |

文件名常见形式为 历史成交\_账号\_日期范围.csv

共有8条

1

50 条/页

# CASE：实盘交易绩效分析

CSV 核心字段（金阳光格式）

| 字段   | 说明      | 示例         |
|------|---------|------------|
| 证券名称 | 股票名称    | 贵州茅台       |
| 证券代码 | 6 位代码   | 600519     |
| 成交日期 | 交易日期    | 2026-02-24 |
| 买卖方向 | 买入 / 卖出 | 卖出         |
| 成交数量 | 股数      | 100        |
| 成交价格 | 成交均价    | 1466.00    |
| 成交金额 | 总金额     | 146600.00  |
| 佣金   | 券商佣金    | 21.99      |
| 印花税  | 卖出侧常见   | 73.30      |
| 过户费  | 沪市等     | 1.47       |

# CASE：实盘交易绩效分析

---

将 6 位代码映射为 xxxxxx.SH / xxxxxx.SZ，无法映射的证券（如港股）在合并后剔除。

```
def code_to_standard(raw_code):
```

```
    code = raw_code.strip()
```

```
    if len(code) == 6:
```

```
        if code.startswith(('6', '5')):
```

```
            return code + '.SH'
```

```
        elif code.startswith(('0', '3')):
```

```
            return code + '.SZ'
```

```
    return None
```

```
# 支持 str 或 list，多文件 concat，按成交编号去重
```

```
trades_df = load_broker_csv(['历史成交_cy_260101-260325.csv'])
```

# CASE：实盘交易绩效分析

## 盈亏、成本、净值与报告

```
OUTPUT_DIR = 'outputs/3-实盘交易分析'

stock_df = analyze_by_stock(trades_df)
cost_info = analyze_costs(trades_df)
nav_series, daily_df = build_portfolio_nav(trades_df,
initial_cash=estimated_cash)

returns = nav_to_returns(nav_series)
metrics = calc_quantstats_metrics(returns)

# 英文版：QuantStats 原版一键生成
generate_html_report(
    returns, title='Real Trading - QuantStats Report',
    output_path=f'{OUTPUT_DIR}/实盘交易_QuantStats报告_英文.html'
)
```

```
# 中文版：自主渲染，中文字体无乱码
generate_chinese_report(
    returns, title='实盘交易 - 绩效分析报告',
    output_path=f'{OUTPUT_DIR}/实盘交易_QuantStats报告.html'
)
# 综合多章节 HTML：generate_trade_analysis_report(...)
```

build\_portfolio\_nav 按成交日更新现金与持仓，每个交易日用 data\_loader.load\_stock\_data 拉取的收盘价对持仓盯市，得到归一化净值序列，再转为日收益率供 QuantStats 使用。

# CASE：实盘交易绩效分析

第一步: 加载券商导出的历史成交记录

读取: 历史成交\_cy\_260101-260325.csv (8 条记录)  
[过滤] 2 条非A股记录 (港股等): 健康元(港)  
合并后: 6 条A股成交记录  
时间范围: 2026-01-20 ~ 2026-03-06

第二步: 个股盈亏分析

| 证券名称  | 买入金额    | 卖出金额      | 未平仓    | 已实现盈亏 | 总成本 |
|-------|---------|-----------|--------|-------|-----|
| 九安医疗  | 148,530 | 0 3000    | 持仓中    | 22    |     |
| 贵州茅台  | 137,950 | 146,600 0 | +8,533 | 117   |     |
| 新天然气  | 81,700  | 81,600 0  | -167   | 67    |     |
| 纳指ETF | 188     | 0 100     | 持仓中    | 5     |     |

已实现盈亏合计: +8,366 元

第三步: 交易成本分析

总成交额: 596,568 元  
佣金: 92 元  
印花税: 114 元  
过户费: 4 元  
总成本: 211 元  
综合费率: 0.035 %

第四步: 构建组合每日净值曲线

净值序列: 28 个交易日

组合收益: +3.78%

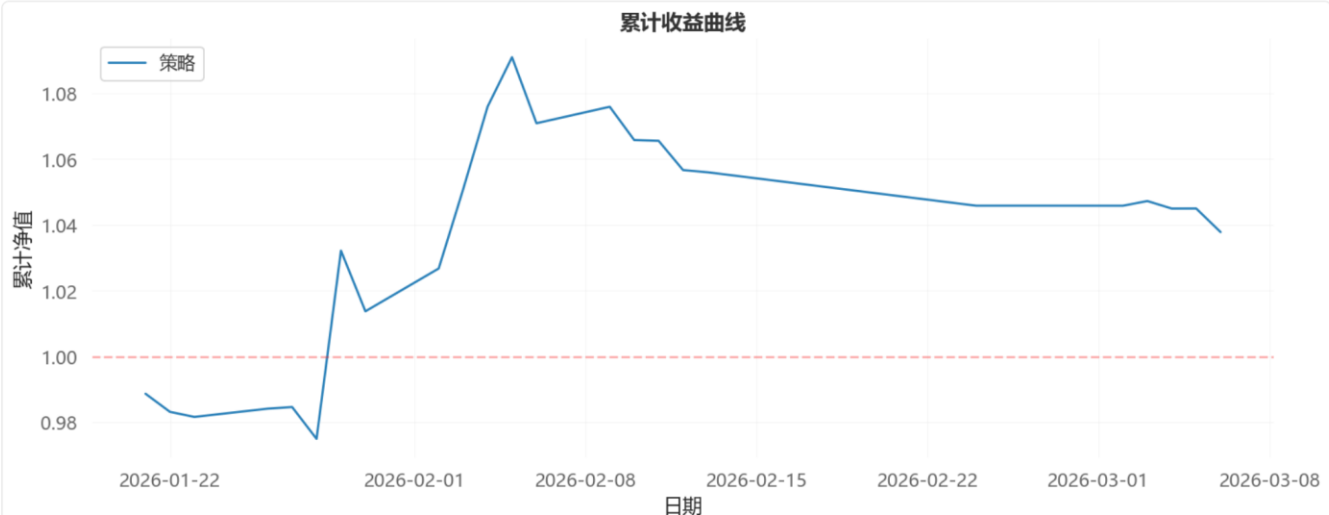
QuantStats 绩效分析:

总收益率 3.78%  
年化收益率(CAGR) 41.41%  
最大回撤 -4.87%  
夏普比率 1.5397  
索提诺比率 3.5142  
卡玛比率 8.4978

# CASE：实盘交易绩效分析



累计收益曲线



日常使用模式:

- 1. 每日收盘后: 运行策略脚本生成次日信号
- 2. 次日开盘前: 根据信号通过 miniQMT 下单
- 3. 定期: 从券商导出CSV, 用本脚本分析实盘绩效

# miniQMT 实盘接口

MiniQMTTrader，封装 xtquant 连接、买卖、查询资产与持仓等

=> 用于与「策略信号 → 程序化下单 → 定期导出 CSV 复盘」链路衔接。

| 方法                                | 说明           |
|-----------------------------------|--------------|
| connect()                         | 连接客户端并订阅资金账号 |
| buy(stock_code, volume, price=0)  | 买入（市价或限价）    |
| sell(stock_code, volume, price)   | 卖出           |
| query_asset() / query_positions() | 资金与持仓        |
| query_today_trades()              | 当日成交         |
| disconnect()                      | 断开           |

# miniQMT 实盘接口

---

```
from importlib import import_module
mod = import_module('3-实盘交易绩效分析')
MiniQMTTrader = mod.MiniQMTTrader

trader = MiniQMTTrader(
    qmt_path=r"D:\\光大证券金阳光QMT实盘\\userdata_mini",
    account_id="你的资金账号"
)
if trader.connect():
    print(trader.query_asset())
    trader.disconnect()
```

成交数据来源：仅通过 QMT / API 下单的成交，未必出现在你最终在券商导出的「全渠道」CSV 中；

反之，手动在 App 下单的成交，也未必进入 miniQMT 导出接口。

以券商导出的历史成交 CSV 作为绩效分析主数据源，可与 API 流水交叉核对。



# CASE：实盘交易绩效分析Plus

# CASE: 实盘交易绩效分析Plus

---

TO DO: 复用 实盘交易绩效分析的逻辑, 在相同 CSV 流程之后增加步骤:

SVD 市场状态诊断, 并生成带 SVD 章节的 实盘交易Plus分析报告.html。

SVD分析原理:

对交易涉及的股票构建收益率矩阵, 进行SVD分解

$$R = U * \text{Sigma} * V^T$$

Sigma中第一个奇异值的方差占比 = 市场一致性程度

占比 > 50%: 齐涨齐跌 | 35%-50%: 板块分化 | < 35%: 个股行情

如果交易数据时间跨度较短(不足SVD窗口), SVD诊断会自动跳过。

建议累计3个月以上的交易数据以获得有效的SVD分析结果。

# CASE：实盘交易绩效分析Plus

## 滚动 SVD 与状态判断

对参与分析的多只股票，拉取区间日行情并构造收益率矩阵；

在长度足够的窗口上反复做 SVD，得到第一奇异值能量占全部奇异值能量和的比例 `top1_var`，并可用前三大奇异值占比 `top3_var` 观察因子集中度。

当前状态通常取最近若干窗口的 `top1_var` 均值，再与 50%、35% 阈值比较。

| 第一因子方差占比  | 市场状态 | 策略取向                   |
|-----------|------|------------------------|
| > 50%     | 齐涨齐跌 | Beta 主导，指数增强、趋势跟随相对更有效 |
| 35% ~ 50% | 板块分化 | 行业轮动、板块配置权重上升          |
| < 35%     | 个股行情 | 选股与 Alpha 模型空间更大       |

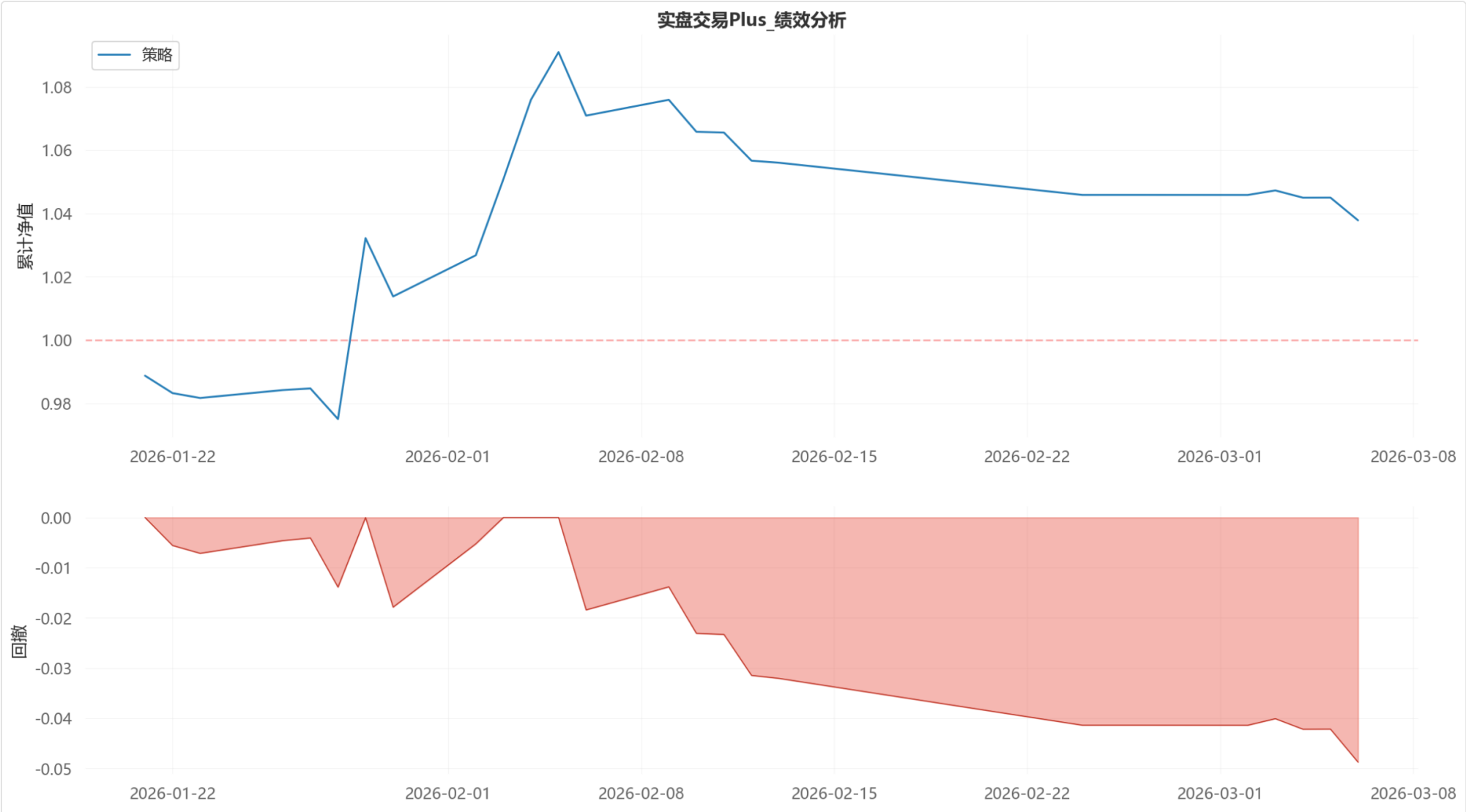
# CASE：实盘交易绩效分析Plus

```
from importlib import import_module
_script3 = import_module('3-实盘交易绩效分析')
load_broker_csv = _script3.load_broker_csv
analyze_by_stock = _script3.analyze_by_stock
build_portfolio_nav = _script3.build_portfolio_nav
analyze_costs = _script3.analyze_costs

# 诊断：默认 window=60, step=10；有效标的不足 3 或交易日不足 window+step 时返回 None
svd_result = diagnose_market_regime(
    stock_codes, start_date, end_date,
    window=60, step=10
)
report_path = generate_plus_report(
    trades_df, stock_df, nav_series, metrics,
    cost_info, charts, svd_result,
    output_path='outputs/4-实盘交易Plus/实盘交易Plus分析报告.html'
)
```

脚本会在 **outputs/4-实盘交易Plus/** 下写入 SVD 趋势图（如 **SVD 市场状态诊断.png**），并在综合报告中嵌入状态卡片、文字建议与滚动窗口因子占比表。

# CASE：实盘交易绩效分析Plus



# CASE：实盘交易绩效分析Plus

## SVD 市场状态诊断

基于 18 只交易标的 150 个交易日的收益率矩阵, 通过滚动SVD分解分析市场因子结构。

当前市场状态  
**个股行情**

Factor1 方差占比: 28.8%

### 投资建议

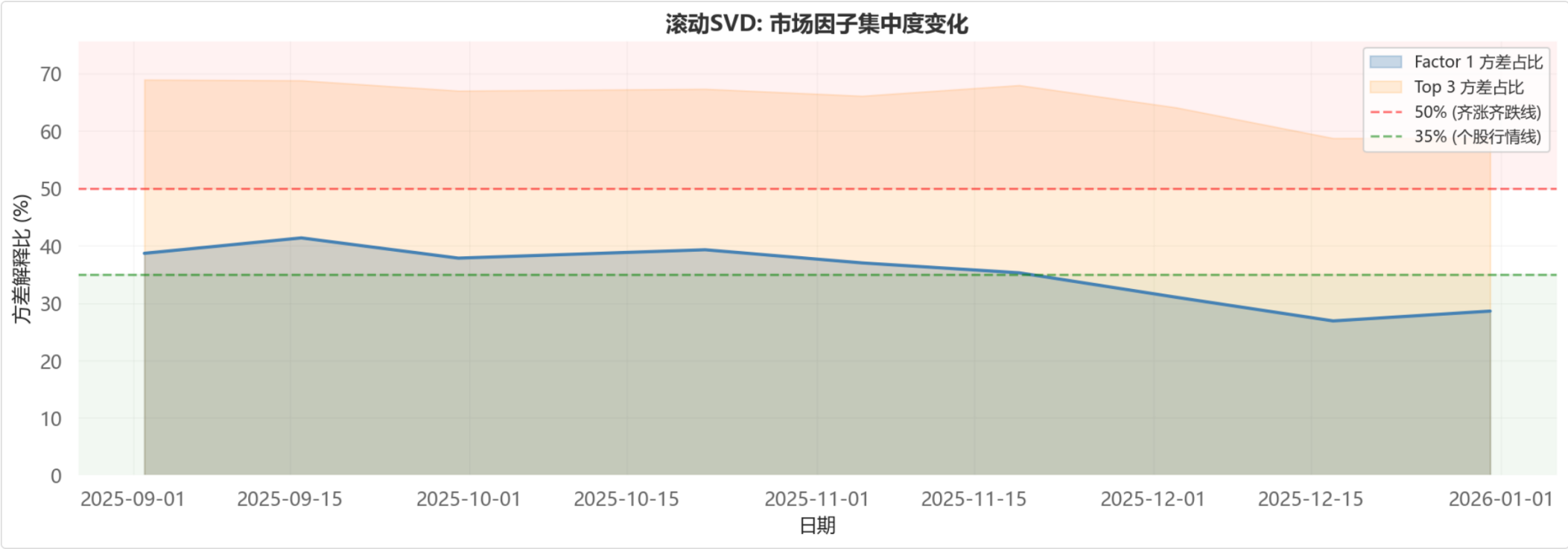
当前市场个股分化明显, alpha机会丰富。建议: 选股策略更有效, 多因子模型价值凸显。可考虑: 精选个股, 降低对大盘方向的依赖。

解读: Factor1方差占比反映市场"一致性"程度 -- > 50% 表示大部分股票同涨同跌(beta主导), <35% 表示个股走势分化(alpha机会多)。

## 滚动SVD因子集中度

| 时间      | Factor1占比 | Top3占比 | 市场状态 |
|---------|-----------|--------|------|
| 2025-09 | 38.7%     | 68.9%  | 板块分化 |
| 2025-09 | 41.4%     | 68.8%  | 板块分化 |
| 2025-09 | 37.8%     | 67.0%  | 板块分化 |
| 2025-10 | 39.3%     | 67.3%  | 板块分化 |
| 2025-11 | 37.0%     | 66.1%  | 板块分化 |
| 2025-11 | 35.3%     | 68.0%  | 板块分化 |
| 2025-12 | 31.0%     | 64.1%  | 个股行情 |
| 2025-12 | 26.9%     | 58.7%  | 个股行情 |
| 2025-12 | 28.6%     | 58.8%  | 个股行情 |

# CASE：实盘交易绩效分析Plus



# 打卡：实盘绩效报告



针对你的实盘交易，输出绩效报告

- 导出历史交易记录 csv
- 使用 QuantStats 进行绩效分析
- 使用 SVD 对你的股票进行分析

你的股票是走独立行情么？还是跟随大盘？





Thank You  
Using data to solve problems